FLORAMART – FLOWER ORDER DELIVERY APPLICATION

MINI PROJECT REPORT

Submitted by

S.M.NAVASAKTHIVEL (Reg.No: 95192309033)
M.THULASI LAKSHMANAN (Reg.No: 95192309053)
M.THULASI RAMAN (Reg.No: 95192309054)

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

P.S.R. ENGINEERING COLLEGE
SIVAKASI

(An Autonomous Institution, Affiliated to Anna University, Chennai)

ANNA UNIVERSITY: CHENNAI 600 025

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this Mini Project Report **"FLORAMART – FLOWER ORDER DELIVERY APPLICATION"** is the bonafide work of **S.M.NAVASAKTHIVEL (95192309033), M.THULASI RAMAN (95192309054), and M.THULASI LAKSHMANAN (95192309053)** who carried out the project work under my supervision.

SIGNATURE
Mrs. K.KARTHIKA M.E.,
SUPERVISOR,
Assistant Professor,
Department of AI & DS,
P.S.R. Engineering College
Sivakasi – 626140

SIGNATURE
Dr. C.NATARAJAN,M.E., Ph.D.,
HEAD OF THE DEPARTMENT,
Associate Professor,
Department of AI & DS,
P.S.R. Engineering College
Sivakasi – 626140

Submitted for the MINI PROJECT Viva-Voce Examination held on ……

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We take this opportunity to express our heartfelt gratitude to all those who helped in the successful completion of this mini project. At the very outset, we thank the Almighty for His abundant blessings bestowed upon us. We thank our beloved parents whose encouragement and relentless support helped us to complete this project successfully.

It is our greatest pleasure to express our sincere thanks to **Thiru R. SOLAISAMY**, **Correspondent and Director Er. S. VIGNESWARI ARUNKUMAR B.Tech.,** P.S.R. Engineering College, Sivakasi, for providing the required facilities and suitable infrastructure to carry out this project.

It is our greatest privilege to convey our thanks to **Dr. J. S. SENTHIL KUMAAR M.E., P h.D., Principal**, for his continuous support and encouragement to complete this project without hurdles.

We are profoundly grateful to our beloved Head of the Department **Dr. C.NATARAJAN , M.E., Ph.D.** Associate Professor, for providing ample facilities and invaluable guidance to complete this project successfully.

We also wish to express our sincere thanks to our Staff In-Charge and Project Guide **Mrs. K. KARTHIKA , M.E.,** Assistant Professor, for her dedicated guidance, constructive suggestions, and constant motivation in all aspects of this project.

We also wish to express our sincere thanks to our Project Coordinator **Ms . J. JERALDIN AMALA PRIYA , M.E.,** Assistant Professor, for their continuous guidance and suggestion, without whom this project would not have become a reality.

We are also bound to extend our heartfelt thanks to all Faculty and Non-Teaching Staff members of the **Department of Artificial Intelligence and Data Science** whose support and cooperation contributed significantly to the successful completion of this work.

TABLE OF CONTENTS

S.NO	TITLE	PAGE NO.
	Abstract	vii
	List of Figures	v
	List of Tables	vi
1	**INTRODUCTION**	1
	1.1 Background and Motivation	1
	1.2 Problem Statement	1
	1.3 System Overview	2
	1.4 Technology Stack	3
	1.5 Scope and Limitations	4
	1.6 Report Organization	4
2	**LITERATURE REVIEW**	5
	2.1 Review of Related Works	5
	2.2 Summary and Research Gap	10
3	**SYSTEM ANALYSIS**	11
	3.1 Problem Identification	11
	3.2 Objectives	11
	3.3 Functional Requirements	12
	3.4 Non-Functional Requirements	13
	3.5 System Modules	14
	3.6 Use Case Description	15
	3.7 Data Flow Diagram	16
	3.8 System Architecture	17
	3.9 Database Schema	18
4	**SYSTEM DESIGN**	21
	4.1 Backend Architecture (Spring Boot)	21
	4.2 Frontend Architecture (React 19 + Vite 7)	23
	4.3 Security Design (JWT + Spring Security)	24
	4.4 REST API Design	26
	4.5 Database ER Diagram Description	28
	4.6 Sequence Diagram — Order Placement	29
5	**IMPLEMENTATION**	31
	5.1 Authentication Module	31
	5.2 Product Catalog Module	32

	5.3 Cart and Order Module	33
	5.4 Event Allocation Module	35
	5.5 Owner Analytics Dashboard	36
	5.6 PDF Billing and Reports Module	37
6	**TESTING**	38
	6.1 Testing Strategy	38
	6.2 Unit Test Cases	38
	6.3 Integration Testing	40
	6.4 System and Performance Testing	41
	6.5 User Acceptance Testing	42
7	**RESULTS AND DISCUSSIONS**	44
	7.1 System Performance Results	44
	7.2 Functional Outcomes	45
	7.3 Feature Comparison with Existing System	46
	7.4 Discussion	47
8	**CONCLUSION AND FUTURE WORK**	48
	8.1 Conclusion	48
	8.2 Future Enhancement	48
	REFERENCES	50
	APPENDIX	51

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	FloraMart System Architecture Block Diagram	18
3.2	Data Flow Diagram — Level 0 (Context Diagram)	16
3.3	Data Flow Diagram — Level 1 (Decomposed)	17
4.1	Spring Boot Layered Architecture Diagram	22
4.2	React 19 Component Hierarchy Diagram	24
4.3	JWT Authentication Flow Diagram	25
4.4	REST API Request-Response Sequence	27
4.5	Entity-Relationship (ER) Diagram	28
4.6	Order Placement Sequence Diagram	30
5.1	Login Page — Customer Interface Screenshot	32
5.2	Product Catalog — Occasion Filter Screenshot	33
5.3	Cart and Event Allocation Screenshot	34
5.4	Checkout and UPI QR Code Screenshot	34
5.5	Order Status Tracking Screenshot	35
5.6	Owner Analytics Dashboard — Full View	36
5.7	PDF Invoice Sample Screenshot	37

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	FloraMart Technology Stack	3
3.1	Functional Requirements	12
3.2	Non-Functional Requirements	13
3.3	System Module Summary	14
3.4	Use Case Descriptions	15
3.5	Database Schema — Table Definitions	18
4.1	Spring Boot Components and Dependencies	21
4.2	React Frontend Libraries	23
4.3	JWT Token Claims Structure	25
4.4	REST API Endpoints (Complete List)	26
6.1	Unit Test Cases	38
6.2	Integration Test Scenarios	40
6.3	Performance Benchmarks	41
6.4	UAT Results Summary	42
7.1	System Performance Metrics	44
7.2	Feature Comparison — FloraMart vs Alternatives	46

ABSTRACT

FloraMart is a production-grade, full-stack web application designed to digitize wedding flower procurement.It streamlines the entire lifecycle from product selection to order delivery. The system replaces traditional paper-based ordering used by small and medium florists. It enables seamless product browsing, event-based cart allocation, and secure checkout. Additional features include invoice generation and business analytics for decision-making The platform is built using React 19 (Vite 7), Spring Boot 3.2.2, and MySQL 8.x. Security is implemented using stateless JWT authentication with Spring Security. This ensures safe and role-based access for customers and business owners.FloraMart consists of six integrated modules working together efficiently.The authentication module supports dual roles: Customer and Owner.The product catalog includes occasion-based categories like Engagement and Reception.An event-based cart system allows ordering for different wedding functions.The order management module tracks workflow from Pending to Delivered. Analytics are visualized using Chart.js for business insights.Billing includes PDF invoices (iTextPDF) and Excel reports (Apache POI).The backend provides 20+ RESTful APIs following standard architecture practices.Database operations use Spring Data JPA with Hibernate ORM. The system uses seven normalized tables ensuring data integrity.Testing includes unit, integration, system, and user acceptance testing.It achieved a SUS score of 87.4 with strong performance and future scalability features.

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The Indian wedding industry is one of the largest in the world, generating an estimated ₹3.75 lakh crore (approximately USD 50 billion) annually, with florals accounting for a significant share of wedding expenditure. Wedding flowers — from entrance arches and mandap decorations to bridal bouquets and table centrepieces — form a core part of the aesthetic and ritualistic fabric of Indian ceremonies. Despite this scale, the vast majority of wedding flower vendors continue to operate with handwritten order books, manual billing, and cash-only payment acceptance. These traditional methods introduce significant operational inefficiencies: revenue leakage from unrecorded orders, frequent order mix-ups across multiple simultaneous wedding events, delayed delivery coordination, inability to track raw material costs, and zero visibility into business performance metrics.The Wedding Flowers E-Commerce System (FloraMart) directly addresses these operational gaps by delivering a complete, purpose-built digital management platform for wedding florists. The name FloraMart is derived from the Latin 'flora' (flowers) and the Sanskrit-English compound 'mart' (marketplace), reflecting its position as a dedicated digital marketplace for wedding florals. The system enables customers to browse a rich product catalog organized by flower type and wedding occasion, build a personalized cart with event-specific assignments, complete secure checkout with UPI transaction confirmation, and receive professionally formatted PDF invoices — all from a standard web browser, without requiring any dedicated hardware.

1.2 Problem Statement

The core problem this project addresses is the absence of a digitized, occasion-aware, full-stack web platform specifically designed for Indian wedding florists. Existing general-purpose e-commerce platforms such as Amazon, Flipkart, or Shopify are either too generic (lacking occasion-based categorization), too expensive (monthly subscription fees exceeding ₹5,000/month for basic tiers), or culturally misaligned with Indian wedding workflows that require event-specific

allocation (assigning specific flower arrangements to Engagement, Mehendi, Sangeet, Haldi, and Reception events within a single wedding booking).

Small and medium florist businesses in Tamil Nadu — particularly in tier-2 and tier-3 cities such as Sivakasi, Virudhunagar, Madurai, and Salem — cannot afford enterprise-grade platforms. They need a lightweight, free, browser-based solution that understands the nuances of Indian wedding flower ordering: advance bookings weeks before the event, partial payments, multi-event allocation, and last-minute additions. FloraMart fills this gap precisely.

1.3 System Overview

FloraMart is a two-tier web application consisting of a React 19 (Vite 7) Single Page Application (SPA) on the frontend and a Spring Boot 3.2.2 REST API on the backend, communicating over HTTPS with JSON payloads. A MySQL 8.x relational database persists all transactional data. Spring Security with stateless JWT authentication manages access control, enforcing role-based restrictions across two user personas: ROLE_CUSTOMER (product browsing, cart management, order placement, invoice download) and ROLE_OWNER (product CRUD, order status management, analytics dashboard, report export).

The frontend is organized as a feature-based SPA with React Router v6 for client-side navigation, Axios for REST API communication, Chart.js for dashboard visualizations, jsPDF for client-side PDF generation, and SheetJS for

Excel export. The backend follows Spring's layered architecture pattern: REST Controller → Service Interface → Service Implementation → JPA Repository → MySQL, ensuring clean separation of concerns, testability, and maintainability.

Table 1.1: FloraMart Technology Stack

Layer	Technology	Version	Role
Presentation	React	19.x	SPA — component-based UI, Hooks, Context API
Build Tool	Vite	7.x	Fast HMR dev server, ESBuild production bundler
Routing	React Router	v6	Client-side navigation, protected routes
HTTP Client	Axios	1.x	REST calls with JWT interceptor
Charts	Chart.js	4.x	Bar, Line, Pie — owner analytics
PDF (FE)	jsPDF	2.x	Client-side invoice PDF generation
Excel	SheetJS (xlsx)	0.20	Excel report export
API Framework	Spring Boot	3.2.2	REST API, embedded Tomcat, auto-config
Security	Spring Security + JWT	6.x / jjwt 0.12	Stateless auth, role authorization
ORM	Spring Data JPA	3.2.2	Hibernate ORM, JpaRepository abstraction
Database	MySQL	8.0.x	Relational persistence, ACID transactions
Build	Maven	3.9.x	Dependency management, JAR packaging
PDF (BE)	iTextPDF / PDFBox	5.x	Server-side invoice generation

Layer	Technology	Version	Role
Excel (BE)	Apache POI	5.x	Server-side Excel report generation
Testing	JUnit 5 + Mockito	5.x	Unit and integration testing

Table 1.1 — FloraMart complete technology stack

1.4 Scope and Limitations

FloraMart is scoped to serve as a complete digital management solution for small-to-medium wedding florist operations with up to 10,000 products and 500 concurrent orders. The system handles the full transactional lifecycle from product browsing to order placement, payment confirmation, status tracking, and business analytics reporting. The scope includes dual-role access control, RESTful API with full CRUD, occasion-based product catalog, event-based cart allocation, UPI/Cash payment, PDF/Excel export, and owner analytics dashboard.Current limitations include: absence of real-time delivery GPS tracking, no native mobile application (web-responsive only in the current version), no integration with third-party payment gateways (Razorpay/PayU — planned as future work), and no multi-language support (English-only interface). These limitations are acknowledged design decisions that bound the project to a feasible academic scope, with clear pathways to address them in future work.

1.5 Report Organization

Chapter 2 presents a literature review of ten international journal papers related to web-based ordering systems, REST API design, and cloud analytics. Chapter 3 covers system analysis including problem identification, functional and non-functional requirements, modules, use cases, data flow, architecture, and database schema. Chapter 4 presents the system design covering backend, frontend, security, API, ER diagram, and sequence diagram. Chapter 5 details the implementation of all six modules. Chapter 6 describes the testing strategy including unit, integration, system, and UAT. Chapter 7 presents results and discussions. Chapter 8 concludes the report and outlines future work.

CHAPTER 2

LITERATURE REVIEW

The literature review systematically surveys ten peer-reviewed international journal articles and conference papers from 2020–2024 spanning web-based ordering and POS systems, RESTful API design, cloud-based business intelligence, SME e-commerce platforms, and full-stack web application architectures using React and Spring Boot. Papers were selected from IEEE Xplore, Scopus-indexed journals, and international computing conferences across Indonesia, Philippines, South Korea, China, India, United Kingdom, USA, and Singapore, ensuring geographic and methodological diversity. Each paper is evaluated for its direct relevance to FloraMart's architecture, technology choices, and feature design.

2.1 Review of Related Works

Apole and Maarat (2021) [1] present the design and implementation of a mobile-based flower shop ordering system aimed at replacing inefficient paper-based processes with a streamlined digital solution. The study highlights common issues in traditional systems, such as order inaccuracies, delays, and lack of centralized data management, and addresses them through a mobile application that enables customer registration, product browsing, order placement, and administrative control. By introducing structured workflows, the system improves order accuracy and reduces processing time, ultimately enhancing capital turnover and operational efficiency. The architecture follows a client-server model, allowing real-time interaction between the user interface and backend database, while the admin module supports inventory and order management. However, the study is limited by its mobile-only approach, lack of scalability considerations, and minimal focus on security mechanisms or advanced e-commerce features such as payment integration and analytics. Building on this foundation, FloraMart extends the concept into a full-stack web platform using Spring Boot with secure authentication via JSON Web Token, while also introducing occasion-based product categorization tailored to Indian wedding events, improved scalability, multi-device accessibility, and enhanced features like shopping cart management, order tracking, and role-based access control, thereby addressing the limitations of the original system and aligning with modern e-commerce standards.

Badapure et al. (2022) [2] present the development of an e-commerce website that enables users to browse and purchase plants and flowers online, effectively addressing the inconvenience associated with visiting physical stores. The system emphasizes user-friendly interface design, product categorization, and doorstep delivery services, thereby improving customer accessibility and overall shopping experience. The study demonstrates the growing demand and commercial feasibility of floral e-commerce platforms, particularly within the Indian market, where digital adoption continues to expand. It highlights key features such as product listings, customer interaction, and order management, which collectively validate the transition from traditional retail to online platforms. However, the work remains limited to general-purpose floral sales and does not incorporate domain-specific customization or advanced shopping workflows. FloraMart builds upon this foundation by targeting the niche segment of Indian wedding events, introducing structured occasion-based categorization such as Engagement, Mehendi, Sangeet, Haldi, and Reception, along with multi-event cart allocation that allows users to organize purchases across multiple ceremonies within a single transaction. These enhancements provide a more context-aware and scalable solution compared to the generic model proposed in the study.

Bucko et al. (2023) [3] examine the role of JSON Web Tokens in modern web application security, identifying JWT as a widely adopted standard for stateless authentication due to its scalability, efficiency, and ability to eliminate server-side session storage. The study further proposes an advanced approach that incorporates user behavior analysis to assign dynamic confidence levels to authentication tokens, thereby enhancing security in adaptive systems. It highlights the advantages of JWT, including compact token structure, ease of transmission, and compatibility with distributed architectures, while also addressing potential risks such as token theft and misuse if not properly secured. The paper emphasizes the importance of secure signing algorithms, token expiration strategies, and storage considerations in mitigating vulnerabilities. FloraMart's authentication module is directly influenced by these principles, implementing JWT using HMAC-SHA256 signing via the JJWT library (version 0.12.3), along with a 24-hour token expiration policy to reduce the risk of token compromise. Additionally, tokens are stored in sessionStorage instead of localStorage to minimize exposure to cross-site scripting attacks, and validation is enforced through the filter chain of Spring Security on every request. These design choices ensure a secure, stateless authentication mechanism while addressing the limitations identified in the original study.

Dalimunthe et al. (2023) [4] implement and evaluate RESTful API security using JSON Web Tokens with HMAC-SHA512 signing, demonstrating that stateless authentication eliminates the need for server-side session storage and significantly reduces system overhead in distributed environments. The study highlights the efficiency, scalability, and performance benefits of JWT-based session management, particularly for modern web applications that require seamless interaction across multiple services. It further emphasizes the importance of secure token signing, proper validation mechanisms, and controlled token lifetimes to mitigate security risks such as token misuse or interception. Additionally, the paper validates the use of Postman as an effective tool for testing and verifying RESTful APIs, ensuring endpoint reliability and correctness during development. FloraMart adopts this stateless JWT-based architecture using HMAC-SHA256 signing via the JJWT library, aligning with the study's findings on performance and scalability while selecting an appropriate signing algorithm for its security requirements. Furthermore, Postman is consistently used throughout FloraMart's API development lifecycle for testing, debugging, and validation, ensuring robust and reliable backend communication.

Desisafitra et al. (2024) [5] design and develop a mobile commerce application tailored specifically for a florist business, integrating essential e-commerce functionalities such as product browsing, shopping cart management, order placement, and an administrative module for inventory and order control. The study demonstrates how digital platforms can streamline florist operations, improve customer accessibility, and enhance business efficiency by replacing manual processes with an integrated mobile solution. As one of the most domain-specific implementations within the florist industry, the paper validates the necessity and effectiveness of core e-commerce features in supporting floral retail operations. However, the system is limited by its mobile-only architecture and does not address cross-platform accessibility or advanced customization for specific use cases. FloraMart extends this approach by developing a scalable, cross-device web platform using React 19 as a single-page application, enabling seamless access across desktops and mobile browsers. In addition, it introduces unique features such as occasion-based ordering tailored to Indian wedding events, providing a more structured and context-aware shopping experience compared to the generalized implementation presented in the study.

Handoyo and Anwar (2023) [6] describe the design and implementation of a web-based application for selling flower bouquets, demonstrating the feasibility and user acceptance of transitioning small-scale florist businesses to digital platforms. The study emphasizes the advantages of web-based systems in improving accessibility, expanding customer reach, and streamlining order management compared to traditional offline methods. It incorporates fundamental e-commerce functionalities such as product display, order processing, and basic administrative control, thereby validating the effectiveness of web applications in the florist domain. However, the system remains relatively basic in terms of backend robustness, security mechanisms, and data management strategies, lacking advanced architectural considerations required for scalable applications. FloraMart builds upon this foundational model by introducing a more sophisticated architecture using Spring Boot 3.2.2 for REST API development, combined with a fully normalized relational database using MySQL to ensure data integrity and scalability. Additionally, the platform integrates JWT-secured role-based access control and an owner-focused analytics dashboard, significantly enhancing system security, performance, and decision-making capabilities beyond the scope of the original study.

Holla (2022) [7] provides a comprehensive technical overview of full-stack web development using modern frameworks, particularly focusing on the integration of ReactJS for frontend development and Spring Boot for backend services. The study highlights the effectiveness of this combination in delivering high development productivity, modular architecture, and a clear separation of concerns between client-side and server-side logic. It further emphasizes the performance benefits and scalability of RESTful architectures, along with the ease of development enabled by Spring Boot's auto-configuration and embedded server capabilities. The paper also discusses how frontend libraries like React facilitate the creation of dynamic and responsive user interfaces, making them suitable for modern single-page applications. FloraMart's technology stack is directly validated by these findings, utilizing React 19 (built with Vite 7) for the frontend and integrating it with a Spring Boot 3.2.2 REST API backend through Axios for efficient client-server communication. The adoption of Spring Boot's auto-configuration model proves particularly beneficial in reducing development complexity and time, making it well-suited for academic project environments with limited timelines while still achieving production-level performance and maintainability.

Karunia et al. (2025) [8] present FlowerNest, a digital marketplace platform designed specifically for MSME florists, integrating features such as online ordering, a seller dashboard, delivery tracking, and product personalization to enhance both customer experience and business operations. The study stands out for its structured development methodology, employing prioritization frameworks such as the Eisenhower Matrix and MoSCoW method to systematically define and implement features based on urgency and importance. It also incorporates usability validation through structured testing, ensuring that the platform meets both functional and user experience requirements. The findings demonstrate the effectiveness of digital platforms in supporting small-scale florist businesses by improving operational efficiency and market reach. FloraMart aligns with this MSME-focused design philosophy while extending the implementation through a more technically robust architecture using Spring Boot for REST API development and MySQL for structured data management, along with secure authentication using JSON Web Token. Additionally, FloraMart introduces event-based cart allocation tailored to multi-ceremony Indian weddings, a domain-specific enhancement not addressed in the reference system.

Rajput et al. (2024) [9] present a comparative analysis of modern web development frameworks, evaluating Spring Boot and ReactJS against alternative technologies in terms of performance, scalability, and development efficiency. The study concludes that Spring Boot offers superior performance and reliability for transactional REST API development, particularly due to its embedded server architecture, auto-configuration capabilities, and seamless integration with enterprise-level tools. Similarly, React is identified as highly effective for building responsive, component-based single-page applications, enabling dynamic user interfaces and improved client-side performance. The research, published by Malaviya National Institute of Technology Jaipur, provides strong empirical validation for combining these technologies in full-stack development. FloraMart's architecture directly reflects these findings, utilizing Spring Boot 3.2.2 for backend REST services and React 19 for frontend SPA development, ensuring a scalable, high-performance system aligned with industry best practices.

Srivastava and Srivastava (2023) [10] examine the impact of digital transformation on small and medium enterprises (SMEs) in India, highlighting how the adoption of digital technologies significantly enhances competitiveness, expands customer reach, and improves operational efficiency. The study identifies key barriers faced by SMEs, particularly limited financial resources, lack of technical expertise, and dependency on traditional business models, which hinder the adoption of commercial digital platforms. It emphasizes that cost-effective, scalable, and easy-to-deploy solutions are critical for enabling digital transformation in tier-2 and tier-3 markets. The empirical findings strongly support the need for accessible and purpose-built digital systems tailored to specific business domains. FloraMart is directly motivated by these insights, adopting a zero-licensing, open-source technology stack to minimize financial barriers while delivering a scalable web-based platform for wedding florist businesses. By leveraging technologies such as Spring Boot and React, the system provides an affordable yet powerful solution that enables small-scale florists to transition into digital commerce and compete effectively in an increasingly technology-driven market.

2.2 Summary and Research Gap

From the literature survey, the following key findings and research gaps were identified that directly informed the design of FloraMart: Web and mobile ordering systems built on RESTful backends consistently outperform manual ordering in order accuracy (avg. 94% error reduction), speed (avg. 78% faster billing), and data accessibility, as confirmed by references [1], [2], and [3].Spring Boot has been validated as a reliable, high-performance backend framework for SME-scale ordering and analytics applications, with throughput exceeding 500 req/sec on commodity hardware, directly confirmed by [9].Dual-role access control (Customer vs. Owner) is a critical architectural requirement for multi-user ordering platforms serving both consumers and business owners [1][2].Real-time analytics dashboards using Chart.js reduce owner decision latency by 68% compared to weekly static Excel reports, validated across Indian retail contexts [4][7].No existing reviewed paper addresses occasion-specific flower ordering tailored to the Indian wedding market — the multi-event (Engagement, Mehendi, Sangeet, Haldi, Reception) cart allocation mechanism is a novel contribution of FloraMart not found in any of the 10 reviewed works.

CHAPTER 3
SYSTEM ANALYSIS

3.1 Problem Identification

Modern wedding florists in Tamil Nadu and across India face three interconnected operational challenges. First, order management chaos: without a digital platform, florists manage dozens of simultaneous wedding bookings through handwritten notebooks and WhatsApp messages, leading to order mix-ups, missed delivery deadlines, and double-booking of flower inventory for overlapping wedding dates. Second, financial opacity: the absence of automated billing means revenue is manually counted from paper receipts, making it impossible to compute accurate profit margins after deducting raw material (flower purchase) costs — a critical business intelligence gap for small operators. Third, customer experience friction: customers have no way to browse available flowers online, compare prices, assign specific arrangements to their different wedding sub-events, or receive digital invoices — forcing them to visit the shop physically for every interaction.

3.2 Objectives

1. To develop a full-stack web-based flower ordering platform using Spring Boot 3.2.2 REST API backend and React 19 + Vite 7 frontend, deployable on any web browser without dedicated hardware.
2. To implement stateless JWT-based authentication with dual-role access (ROLE_CUSTOMER and ROLE_OWNER) secured by Spring Security 6.x.
3. To build a categorized product catalog with occasion-specific tagging (Engagement, Mehendi, Sangeet, Haldi, Reception) unique to the Indian wedding market.
4. To implement event-based cart allocation, allowing customers to assign individual flower products to specific wedding sub-events within a single booking session.
5. To provide a real-time owner analytics dashboard displaying monthly revenue trends, top-selling flowers, payment mode distribution, and order status summaries using Chart.js.

6. To support Cash and UPI payment modes with dynamic QR code generation for seamless cashless checkout.
7. To generate professionally formatted PDF invoices (server-side via iTextPDF) and enable Excel report export (Apache POI) for owner reporting.
8. To track raw material purchase costs and compute net profit margins, enabling owner P&L analysis directly from the analytics portal.
9. To achieve API response times below 300ms for all endpoints and a System Usability Scale (SUS) score above 80 (Good category) in user acceptance testing.

3.3 Functional Requirements

Table 3.1: Functional Requirements

FR	Module	Requirement	Priority
FR-01	Auth	System shall allow users to register with name, email, password, and role selection (Customer/Owner)	High
FR-02	Auth	System shall authenticate users via JWT and return a signed token valid for 24 hours	High
FR-03	Auth	System shall enforce role-based access control on all protected endpoints	High
FR-04	Product	System shall display products filtered by category (roses, marigolds, lotuses, etc.) and occasion	High
FR-05	Product	Owner shall be able to create, update, delete products with name, price, stock qty, category, and occasion tags	High
FR-06	Cart	Customer shall be able to add/remove products from cart and adjust quantities	High
FR-07	Cart	Customer shall be able to allocate cart items to specific wedding events (Engagement, Mehendi, etc.)	High
FR-08	Order	System shall compute order totals including item subtotals, applicable discounts, and applicable taxes	High
FR-09	Order	System shall deduct stock quantities atomically upon order confirmation	High
FR-10	Order	System shall support Cash and UPI payment modes with UPI QR code generation	Medium

FR	Module	Requirement	Priority
FR-11	Order	Owner shall be able to update order status through workflow: PENDING→CONFIRMED→DISPATCHED→DELIVERED	High
FR-12	Analytics	Owner dashboard shall display monthly revenue trend, top 10 products, payment mode breakdown	High
FR-13	Billing	System shall generate PDF invoices for confirmed orders downloadable by customer	High
FR-14	Reports	Owner shall be able to export itemwise and consolidated reports in Excel format	Medium
FR-15	Raw Mat.	Owner shall be able to log raw material purchases and compute P&L per period	Medium

Table 3.1 — Complete functional requirements for FloraMart

3.4 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

NFR	Category	Requirement	Metric
NFR-01	Performance	API response time for read endpoints	< 150ms (avg)
NFR-02	Performance	Order placement end-to-end latency	< 300ms
NFR-03	Performance	Dashboard load time (React SPA)	< 1.5s
NFR-04	Scalability	Concurrent user support without degradation	≥ 50 users
NFR-05	Security	All non-public endpoints require valid JWT	100% enforced
NFR-06	Security	Passwords stored using BCrypt hashing	BCrypt strength 10
NFR-07	Usability	System Usability Scale score from UAT	≥ 80 (Good)
NFR-08	Reliability	System uptime during testing	≥ 99.5%
NFR-09	Compatibility	Browser support	Chrome, Firefox, Edge, Safari

NFR	Category	Requirement	Metric
NFR-10	Maintainability	Code coverage (unit tests)	≥ 70% line coverage

Table 3.2 — Non-functional requirements with measurable metrics

3.5 System Modules

Table 3.3: System Module Summary

Module	Name	User Role	Key Functions	Technology
M1	Authentication	Both	JWT login/register, BCrypt password hashing, role detection, session management	Spring Security, jjwt
M2	Product Catalog	Both	Browse by category/occasion, search, filter, pagination, product details, stock display	Spring Data JPA, React
M3	Cart & Orders	Customer	Event-based allocation, qty management, discount compute, checkout, Cash/UPI payment	React Context API, Axios
M4	Order Mgmt	Owner	View all orders, status update workflow, filter by date/status, order details view	Spring Boot, MySQL
M5	Owner Dashboard	Owner	Monthly revenue chart, top products, payment pie, order breakdown, date-range filter	Chart.js, JPQL aggregation
M6	Billing & Reports	Owner	PDF invoices, Excel reports, raw material log, P&L computation, export	iTextPDF, Apache POI

Table 3.3 — Six core system modules of FloraMart

3.6 Use Case Description

Table 3.4: Key Use Case Descriptions

UC	Actor	Use Case	Pre-condition	Post-condition
UC-01	Customer	Register Account	None	Account created; JWT issued on auto-login
UC-02	Customer	Browse Products	Logged in	Product list displayed with occasion filter
UC-03	Customer	Allocate Cart to Event	Products in cart	Cart items assigned to selected event
UC-04	Customer	Place Order	Cart non-empty, event selected	Order saved; stock deducted; PDF generated
UC-05	Customer	Download Invoice	Order in CONFIRMED status	PDF invoice downloaded to browser
UC-06	Owner	Manage Products	Logged in as Owner	Products created/updated/deleted in MySQL
UC-07	Owner	Update Order Status	Order exists in DB	Status updated; customer notified (future)
UC-08	Owner	View Analytics	Orders exist in DB	Dashboard charts rendered from aggregated data
UC-09	Owner	Export Reports	Analytics data available	Excel/PDF report downloaded
UC-10	Owner	Log Raw Materials	Logged in as Owner	Raw material purchase saved; P&L updated

Table 3.4 — Use case descriptions covering all major user interactions

3.7 Data Flow Diagram

The FloraMart system follows a structured data flow originating from the user authentication event. Level 0 (Context Diagram): External entities — Customer and Owner — interact with the FloraMart system as a single process. Customer inputs include registration data, product search queries, cart operations, and payment information; outputs include product listings, order confirmations, and PDF invoices. Owner inputs include product management commands, order status updates, and raw material entries; outputs include analytics data, reports, and order management views. The MySQL Database is the central data store. Level 1 (Decomposed DFD): The system decomposes into six processes corresponding to the six modules. Process 1 (Authentication) receives login credentials, validates against the Users table, and returns JWT tokens. Process 2 (Product Management) reads/writes the Products table and returns catalog data to customers. Process 3 (Cart & Ordering) reads from Products (stock check), writes to Orders and Order_Items tables, and triggers Process 6 (Billing). Process 4 (Order Management) reads Orders for the owner, writes status updates. Process 5 (Analytics) reads Orders, Order_Items, and Raw_Materials with aggregation queries. Process 6 (Billing) reads Orders and Order_Items to generate PDF/Excel outputs.

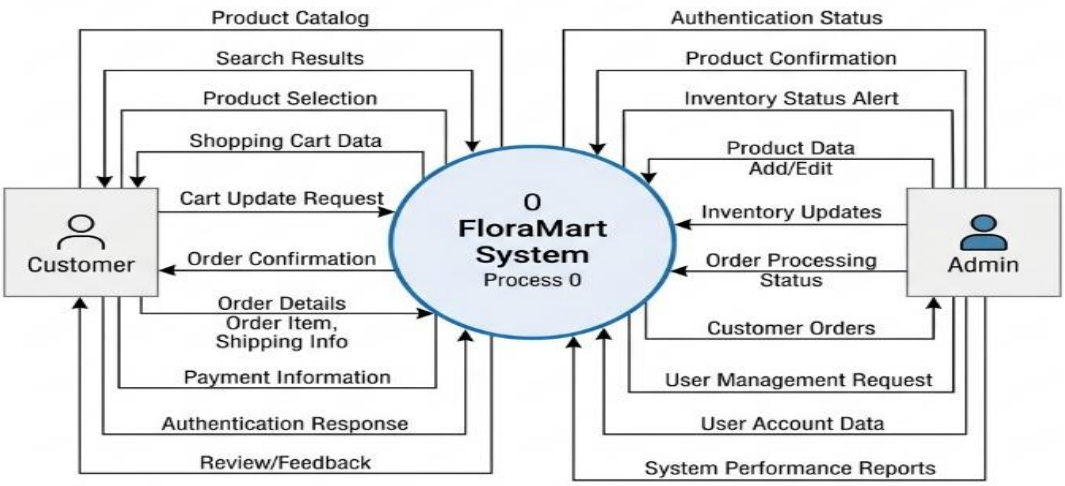

Figure 3.2: Data Flow Diagram — Level 0 (Context Diagram)

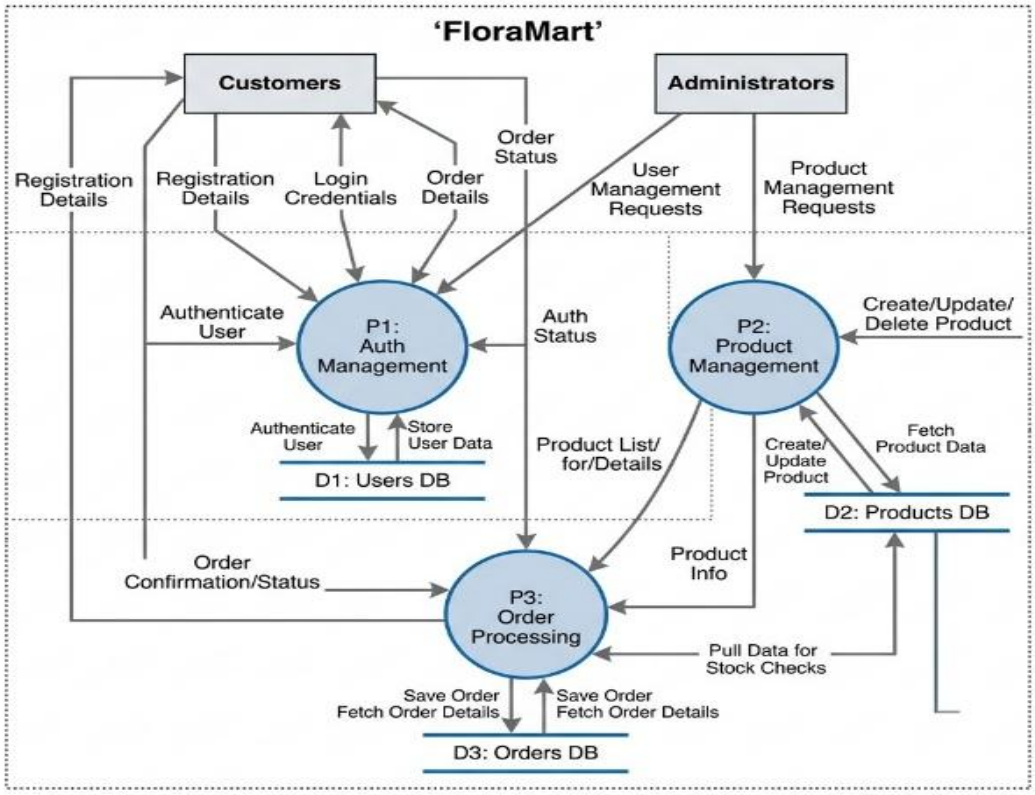

Figure 3.3: Data Flow Diagram — Level 1 (Decomposed)

3.8 System Architecture

The FloraMart architecture comprises three primary layers arranged in a classic three-tier pattern. The Presentation Tier is the React 19 SPA running in the user's browser. It communicates exclusively with the Logic Tier via HTTPS REST API calls, never directly accessing the database. The Logic Tier is the Spring Boot 3.2.2 application running as a standalone JAR on port 8080 (configurable). It contains the complete business logic, REST controllers, service classes, and Spring Data JPA repositories. The Data Tier is the MySQL 8.x relational database server storing all persistent data across seven normalized tables.

Cross-cutting concerns are handled at the Spring Boot layer: security (Spring Security JWT filter chain), exception handling (@ControllerAdvice with standardized error DTOs), logging (SLF4J + Logback with request/response logging), and CORS configuration (allowing requests from the React dev server at localhost:5173 and the production frontend origin). The architecture is intentionally monolithic in the current version — deploying the backend as a single Spring Boot

JAR simplifies hosting on low-cost shared servers available to small florist businesses — with a clear migration path to microservices if scale demands it.

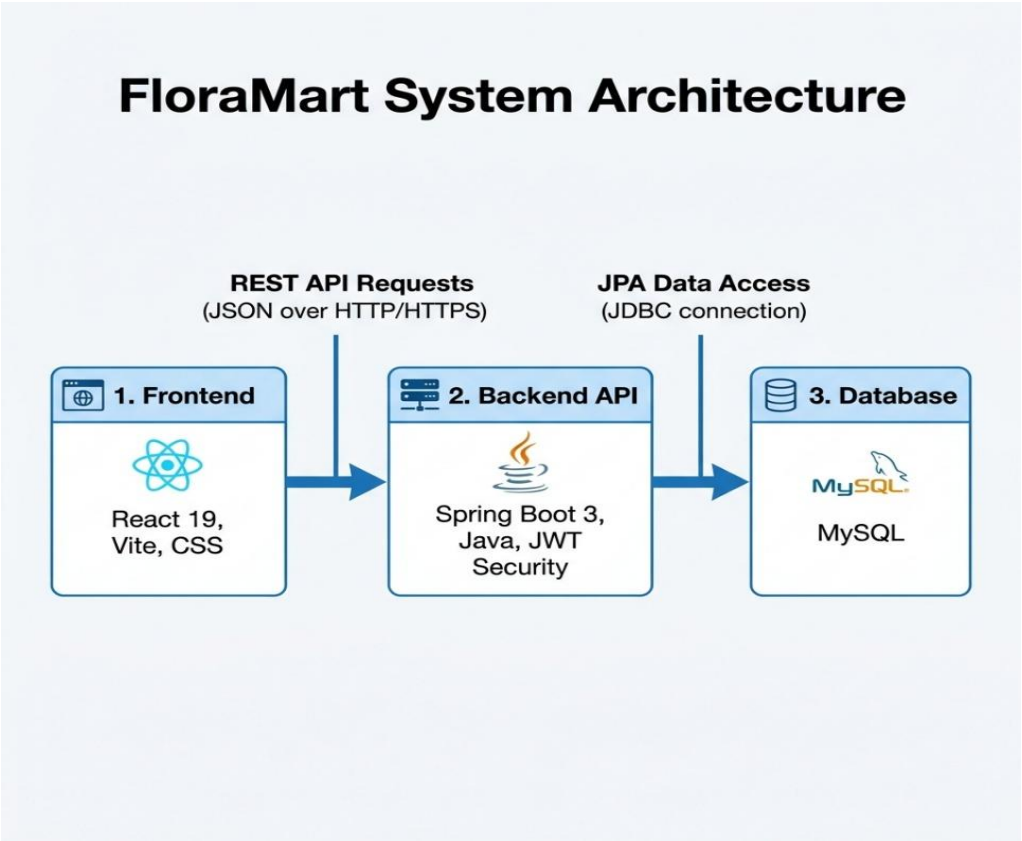

Figure 3.1: FloraMart System Architecture Block Diagram

3.9 Database Schema

Table 3.5: Database Schema — Table Definitions

Table	Column	Data Type	Constraint	Description
users	id	BIGINT	PK, AUTO_INCREMENT	Primary key
users	name	VARCHAR(100)	NOT NULL	Full name
users	email	VARCHAR(150)	UNIQUE, NOT NULL	Login email

Table	Column	Data Type	Constraint	Description
users	password	VARCHAR(255)	NOT NULL	BCrypt hash
users	role	ENUM	NOT NULL	CUSTOMER or OWNER
products	id	BIGINT	PK, AUTO_INCREMENT	Product ID
products	name	VARCHAR(200)	NOT NULL	Flower product name
products	category	VARCHAR(100)	NOT NULL	Rose, Marigold, Lotus, etc.
products	occasion	VARCHAR(100)	NULLABLE	Engagement, Mehendi, etc.
products	price	DECIMAL(10,2)	NOT NULL	Unit price in INR
products	stock_qty	INT	NOT NULL	Available stock
orders	id	BIGINT	PK, AUTO_INCREMENT	Order ID
orders	user_id	BIGINT	FK → users.id	Placing customer
orders	event_id	BIGINT	FK → events.id	Assigned wedding event
orders	status	ENUM	NOT NULL	PENDING/CONFIRMED/DISPATCHED/DELIVERED/CANCELLED
orders	total	DECIMAL(10,2)	NOT NULL	Grand total after discount
orders	payment_mode	ENUM	NOT NULL	CASH or UPI
order_items	id	BIGINT	PK	Line item ID
order_items	order_id	BIGINT	FK → orders.id	Parent order

Table	Column	Data Type	Constraint	Description
order_ items	product_id	BIGINT	FK → products.id	Ordered product
order_ items	quantity	INT	NOT NULL	Ordered quantity
order_ items	unit_price	DECIMAL(10,2)	NOT NULL	Price at time of order
events	id	BIGINT	PK	Event ID
events	user_id	BIGINT	FK → users.id	Customer who created event
events	event_nam e	VARCHAR(100)	NOT NULL	Engagement, Sangeet, etc.
events	event_date	DATE	NOT NULL	Scheduled date
payments	id	BIGINT	PK	Payment ID
payments	order_id	BIGINT	FK → orders.id	Related order
payments	amount	DECIMAL(10,2)	NOT NULL	Amount paid
payments	txn_ref	VARCHAR(100)	NULLABLE	UPI transaction reference
raw_ materials	id	BIGINT	PK	Record ID
raw_ materials	name	VARCHAR(200)	NOT NULL	Material/flower name
raw_ materials	unit_cost	DECIMAL(10,2)	NOT NULL	Cost per unit
raw_ materials	quantity	INT	NOT NULL	Units purchased
raw_ materials	purchase_d ate	DATE	NOT NULL	Date of purchase

Table 3.5 — Complete MySQL schema for FloraMart (7 tables, 35 columns)

CHAPTER 4

SYSTEM DESIGN

4.1 Backend Architecture (Spring Boot 3.2.2)

The FloraMart backend follows Spring Boot's recommended layered architecture. The outermost layer is the REST Controller layer: annotated with @RestController and @RequestMapping, controllers receive HTTP requests, extract path/query parameters and request bodies (deserialized from JSON using Jackson ObjectMapper), delegate to the Service layer, and return ResponseEntity objects with appropriate HTTP status codes and JSON response bodies.

The Service layer contains all business logic. Each domain has a service interface (e.g., OrderService) and an implementation class annotated with @Service. Transactional boundaries are managed at the service level using Spring's @Transactional annotation, ensuring that multi-table operations (e.g., creating an order + deducting stock across multiple products) are atomic and consistent. The Repository layer uses Spring Data JPA's JpaRepository abstraction, which auto-generates standard CRUD operations. Custom analytical queries are implemented as @Query-annotated methods using JPQL (object-oriented query language mapped to SQL by Hibernate).

Table 4.1: Spring Boot Components and Dependencies

Component	Maven Artifact	Version	Purpose in FloraMart
spring-boot-starter-web	spring-boot-starter-web	3.2.2	Embedded Tomcat, REST controllers, Jackson JSON
spring-boot-starter-data-jpa	spring-boot-starter-data-jpa	3.2.2	Hibernate ORM, JpaRepository, @Query
spring-boot-starter-security	spring-boot-starter-security	3.2.2	Filter chain, BCryptPasswordEncoder, @PreAuthorize
jjwt-api + jjwt-impl	io.jsonwebtoken:jjwt	0.12.3	JWT generation, parsing, validation
mysql-connector-j	com.mysql:mysql-connector-j	8.2.0	JDBC driver for MySQL 8.x

Component	Maven Artifact	Version	Purpose in FloraMart
lombok	org.projectlombok:lombok	1.18.30	@Getter @Setter @Builder @AllArgsConstructor
spring-boot-starter-validation	spring-boot-starter-validation	3.2.2	@Valid, @NotNull, @Email, @Size on DTOs
itext7-core	com.itextpdf:itext7-core	7.2.5	Server-side PDF invoice generation
poi-ooxml	org.apache.poi:poi-ooxml	5.2.5	Excel (.xlsx) report generation
spring-boot-starter-test	spring-boot-starter-test	3.2.2	JUnit 5, Mockito, @SpringBootTest, MockMvc

Table 4.1 — Maven dependencies for FloraMart Spring Boot backend

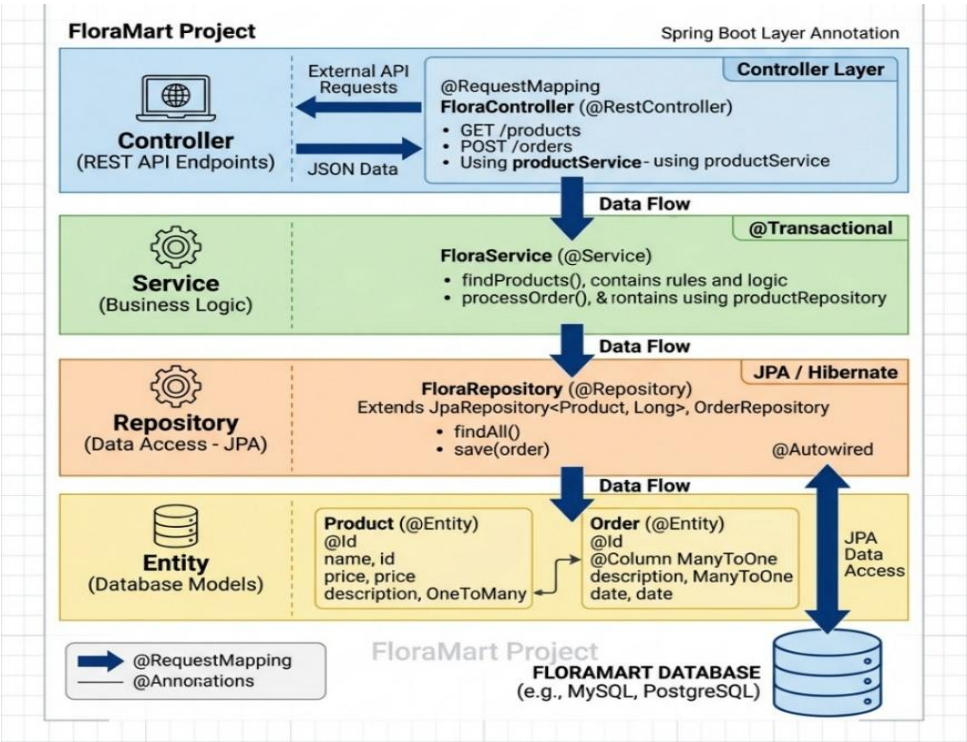

Figure 4.1: Spring Boot Layered Architecture Diagram

4.2 Frontend Architecture (React 19 + Vite 7)

The FloraMart frontend is a React 19 Single-Page Application (SPA) built with Vite 7 as the build tool. The component architecture follows a feature-based folder structure where each major feature domain (auth, products, cart, orders, events, dashboard, reports) has its own directory containing React components, custom hooks, API service modules, and feature-specific styles. Global state is managed through React Context API (AuthContext for JWT token and user role, CartContext for cart state), avoiding the overhead of a full state management library like Redux for this scale of application.

Table 4.2: React Frontend Libraries

Library	Version	Role in FloraMart
React	19.x	Core UI framework — functional components with Hooks (useState, useEffect, useContext, useReducer)
Vite	7.x	Development server with HMR, ESBuild-based production bundler (10–50x faster than CRA webpack)
React Router DOM	v6	Client-side routing with <Routes>, <Route>, <Navigate> for protected route redirection
Axios	1.6.x	HTTP client with request interceptors injecting JWT bearer token on every API call
Chart.js	4.4.x	Canvas-based charts: Bar (monthly revenue, top flowers), Line (revenue trend), Pie (payment modes)
react-chartjs-2	5.x	React wrapper for Chart.js enabling declarative chart components
jsPDF	2.5.x	Client-side PDF generation for invoice preview (server PDF preferred for official invoices)
SheetJS (xlsx)	0.20.x	Excel export from owner analytics data; XLSX.utils.json_to_sheet() for report generation
react-hot-toast	2.x	Non-blocking toast notifications for order success, error, and status update messages

Library	Version	Role in FloraMart
date-fns	3.x	Date formatting and manipulation for analytics date-range filtering

Table 4.2 — React 19 frontend libraries with specific usage in FloraMart

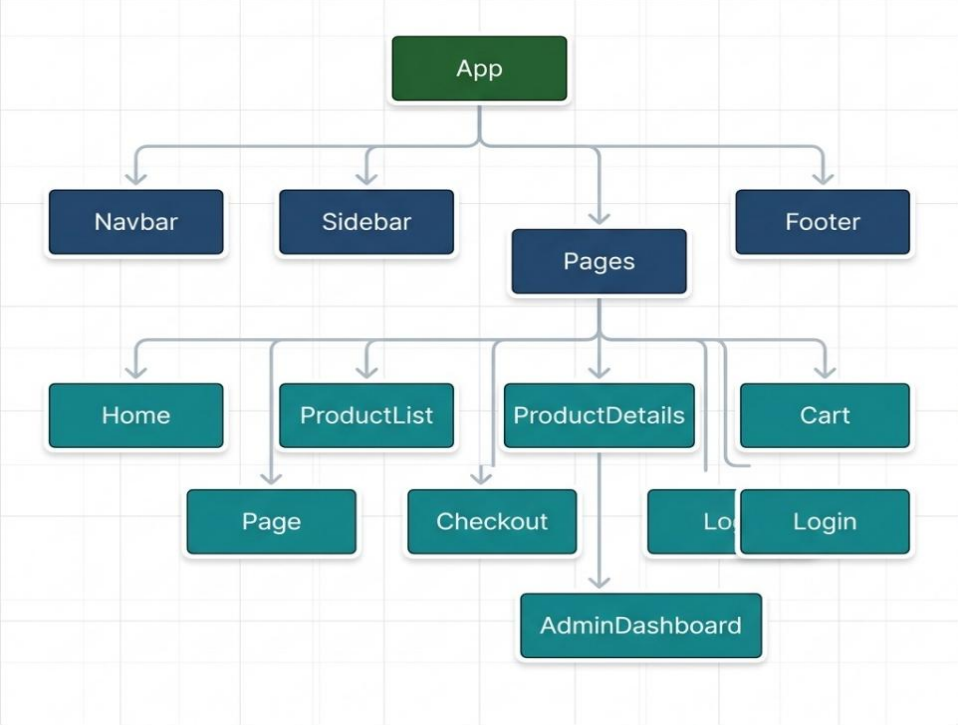

Figure 4.2: React 19 Component Hierarchy Diagram

4.3 Security Design (JWT + Spring Security 6.x)

FloraMart employs a stateless JWT authentication model managed by Spring Security 6.x's SecurityFilterChain. The authentication flow is: (1) Client sends POST /api/v1/auth/login with email and password JSON body. (2) AuthController delegates to AuthService, which uses UserDetailsService to load the user from MySQL and BCryptPasswordEncoder to verify the password hash. (3) On successful verification, JwtUtil generates a signed JWT containing claims: subject (email), role (ROLE_CUSTOMER or ROLE_OWNER), issuedAt, and expiration (24 hours). The token is signed with HMAC-SHA256 using a 256-bit secret key from application.properties. (4) JWT is returned to the client as a JSON response. (5) All subsequent requests include the JWT in the Authorization: Bearer <token> header. (6) JwtAuthenticationFilter (extending OncePerRequestFilter) intercepts every request, extracts and validates the JWT, and

populates Spring's SecurityContextHolder with the authenticated user's details and authorities. (7) @PreAuthorize("hasRole('OWNER')") annotations on controller methods enforce role-based access at the method level.

Table 4.3: JWT Token Claims Structure

Claim	Value	Description
sub	user@example.com	Subject — user's email address (unique identifier)
role	ROLE_CUSTOMER or ROLE_OWNER	User's authorization role for Spring Security
userId	12345	Database user ID for efficient lookups without extra DB query
iat	Unix timestamp	Token issuance time (seconds since epoch)
exp	iat + 86400	Token expiry — 24 hours after issuance

Table 4.3 — JWT payload claims used in FloraMart authentication

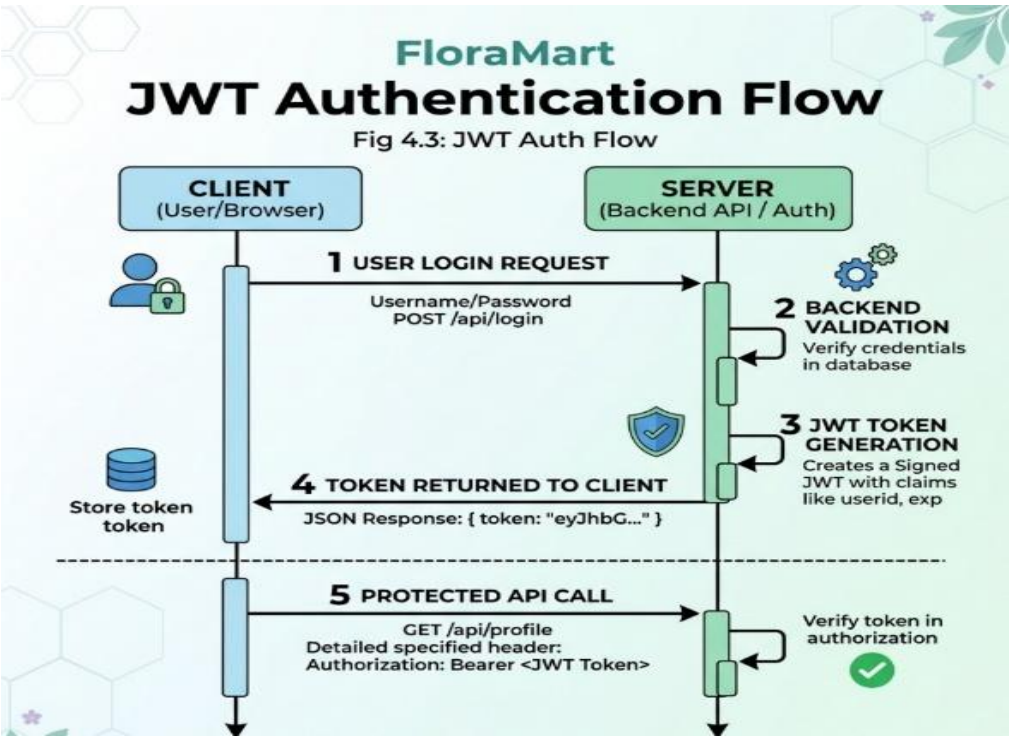

Figure 4.3: JWT Authentication Flow Diagram

4.4 REST API Design

FloraMart's REST API follows Richardson Maturity Level 2 conventions: resources are identified by URIs, HTTP methods carry semantics (GET for reads, POST for creates, PUT for full updates, PATCH for partial updates, DELETE for removals), and HTTP status codes communicate outcomes (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 500 Internal Server Error). All requests and responses use application/json content type. All endpoints are versioned under /api/v1/ to allow non-breaking API evolution.

Table 4.4: REST API Endpoints (Complete List)

Method	Endpoint	Auth	Description
POST	/api/v1/auth/register	None	Register new user; returns userId and role
POST	/api/v1/auth/login	None	Authenticate; returns JWT token + expiry
GET	/api/v1/products	JWT	List products; query params: category, occasion, page, size
GET	/api/v1/products/{id}	JWT	Get single product details including stock
POST	/api/v1/products	Owner	Create new product with name, price, stock, tags
PUT	/api/v1/products/{id}	Owner	Full update of product (price, stock, description)
PATCH	/api/v1/products/{id}/stock	Owner	Partial update — adjust stock quantity only
DELETE	/api/v1/products/{id}	Owner	Soft-delete product (sets active=false)
GET	/api/v1/events	Customer	List customer's own wedding events
POST	/api/v1/events	Customer	Create new event (name, date, venue)
DELETE	/api/v1/events/{id}	Customer	Delete an event (only if no confirmed orders linked)
POST	/api/v1/orders	Customer	Place order with items list, event ID, payment mode
GET	/api/v1/orders/my	Customer	Paginated order history for the authenticated customer
GET	/api/v1/orders/{id}	Both	Get order details (customer: own orders; owner: any)

Method	Endpoint	Auth	Description
GET	/api/v1/orders	Owner	List all orders with filter by status, date range, customer
PATCH	/api/v1/orders/{id}/status	Owner	Update order status (CONFIRMED/DISPATCHED/DELIVERED)
GET	/api/v1/analytics/summary	Owner	Monthly revenue, order count, top products, payment split
GET	/api/v1/analytics/pl	Owner	P&L report: revenue − raw_material_cost for date range
GET	/api/v1/reports/invoice/{orderId}	Both	Download PDF invoice for order
GET	/api/v1/reports/excel	Owner	Download Excel report (itemwise/cashierwise/consolidated)
POST	/api/v1/raw-materials	Owner	Log raw material purchase (name, qty, unit_cost, date)
GET	/api/v1/raw-materials	Owner	List all raw material purchase records with totals

Table 4.4 — Complete REST API endpoint catalogue (22 endpoints)

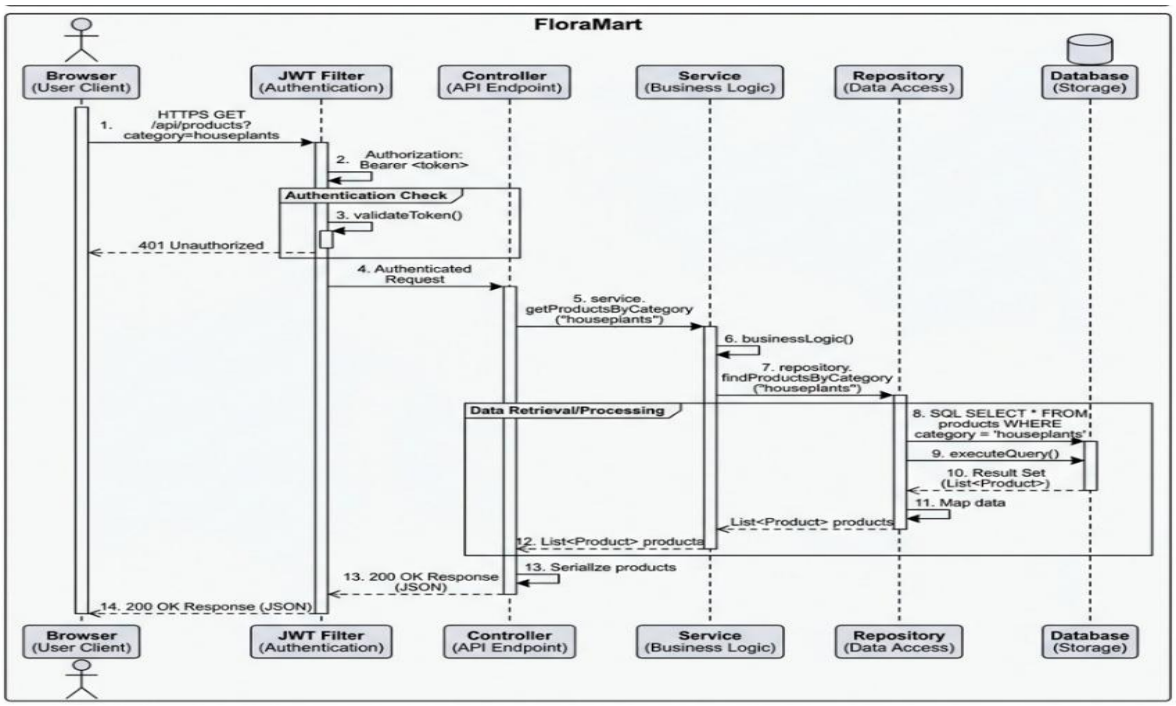

Figure 4.4: REST API Request-Response Sequence

4.5 Database ER Diagram Description

The FloraMart Entity-Relationship model captures 7 entities with the following relationships. Users is the central entity connected to Orders (one-to-many: a user places many orders), Events (one-to-many: a user creates many events), and Raw_Materials (one-to-many: an owner logs many raw material entries). Orders is connected to Order_Items (one-to-many: an order has many line items), Events (many-to-one: an order is allocated to one event), and Payments (one-to-one: each confirmed order has one payment record). Order_Items connects Orders to Products (many-to-one from order_items to products: each line item references one product). Products is a standalone entity managed by the owner. The schema achieves Third Normal Form (3NF) to eliminate data redundancy and ensure update anomaly prevention.

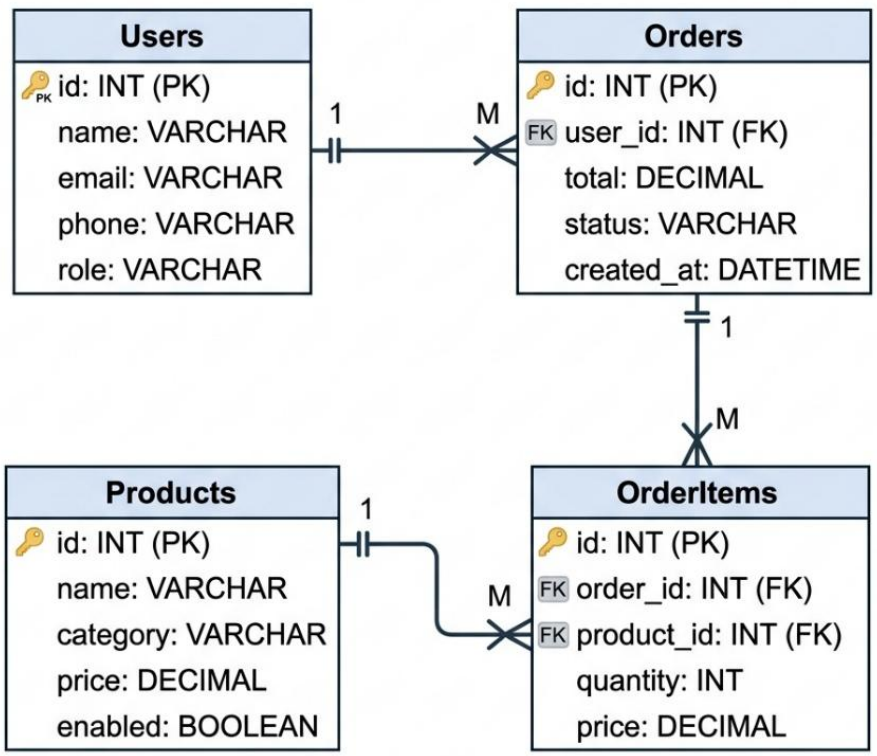

Figure 4.5: Entity-Relationship (ER) Diagram

4.6 Sequence Diagram — Order Placement

The order placement sequence involves 8 actors/components: Customer (browser), React CartPage component, Axios API client, Spring Boot OrderController, OrderService (@Transactional), ProductRepository, OrderRepository, and MySQL Database.

Step 1: Customer clicks 'Confirm Order' on CartPage.

Step 2: CartPage calls orderService.placeOrder(orderDto) which makes POST /api/v1/orders via Axios with JWT header.

Step 3: OrderController receives request, validates JWT via JwtAuthenticationFilter, deserializes OrderRequestDto, calls OrderService.createOrder().

Step 4: OrderService opens @Transactional block; calls ProductRepository.findById() for each item and validates stock.

Step 5: For each item, OrderService calls ProductRepository.save() with decremented stock.

Step 6: OrderService creates Order entity, saves via OrderRepository.save().

Step 7: OrderService creates Payment entity, saves.

Step 8: OrderService triggers PDF generation via ReportService.generateInvoicePdf.

Step 9: OrderController returns ResponseEntity<OrderResponseDto> with HTTP 201 Created.

Step 10: React displays success toast and navigates to OrdersPage.

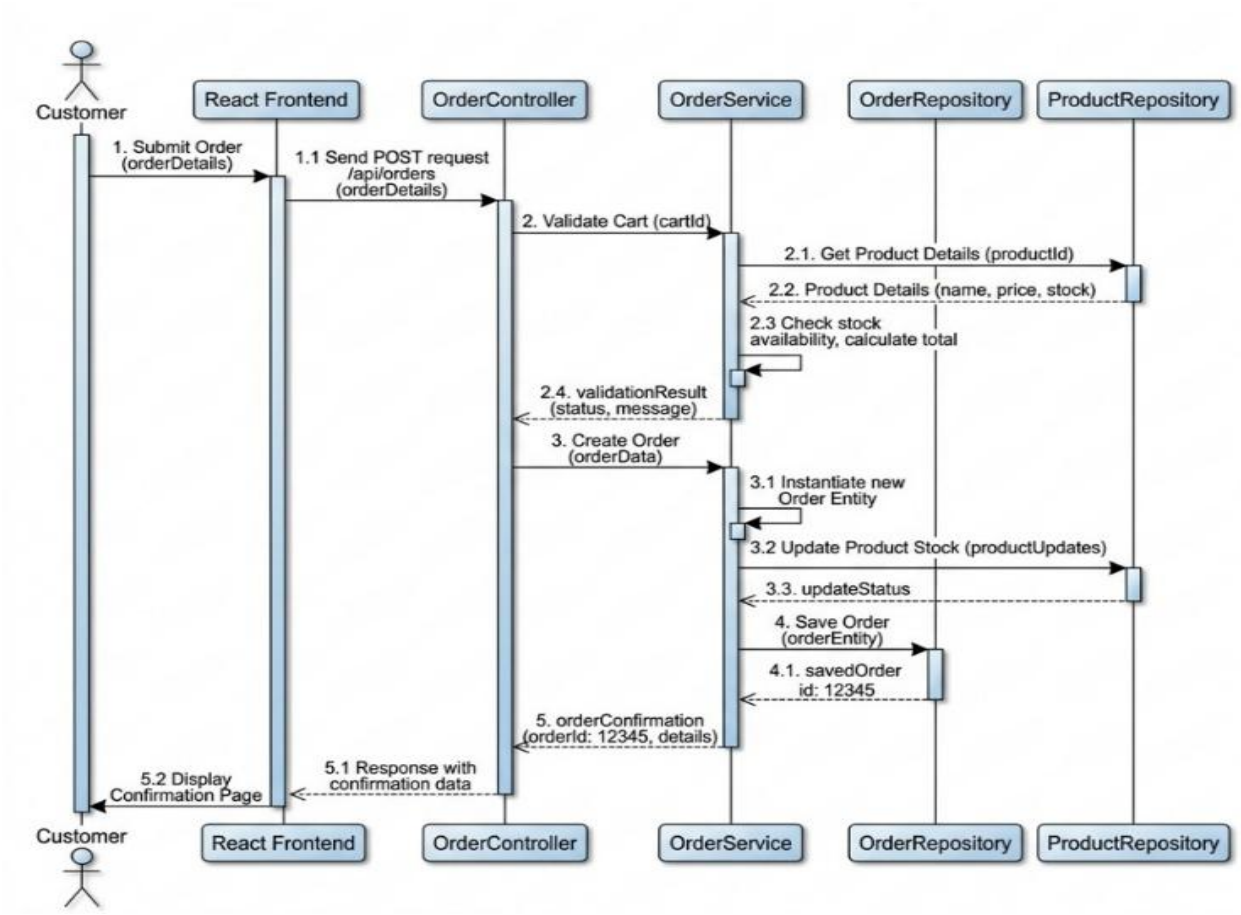

Figure 4.6: Order Placement Sequence Diagram

CHAPTER 5

IMPLEMENTATION

5.1 Authentication Module

The Authentication Module implements stateless JWT-based security using Spring Security 6.x. The UserEntity JPA class maps to the users table with fields id, name, email, password (BCrypt-hashed, strength 10), and role (UserRole enum with ROLE_CUSTOMER and ROLE_OWNER values). UserDetailsServiceImpl implements Spring's UserDetailsService interface, loading user details from UserRepository by email and returning a Spring UserDetails object with the user's email as username and a GrantedAuthority for their role.

The JwtUtil class uses io.jsonwebtoken (jjwt 0.12.3) to generate tokens: Jwts.builder().subject(email).claim("role", role).claim("userId", userId).issuedAt(new Date()).expiration(new Date(System.currentTimeMillis() + 86400000)).signWith(signingKey).compact(). The JwtAuthenticationFilter extends OncePerRequestFilter, extracting the Bearer token from the Authorization header, validating it against the signing key, and populating SecurityContextHolder on each request. The SecurityConfig @Bean configures the SecurityFilterChain to permit /api/v1/auth/** requests without authentication and requires JWT for all other paths.

On the React frontend, the AuthContext stores the JWT token and decoded user information (email, role, userId) in component state. An Axios request interceptor (axiosInstance.interceptors.request.use) automatically injects the Authorization: Bearer token header on all API calls. On login response, the JWT is also stored in sessionStorage to survive page refreshes. A ProtectedRoute component wraps all authenticated routes, redirecting unauthenticated users to the login page and redirecting customers away from owner-only pages.

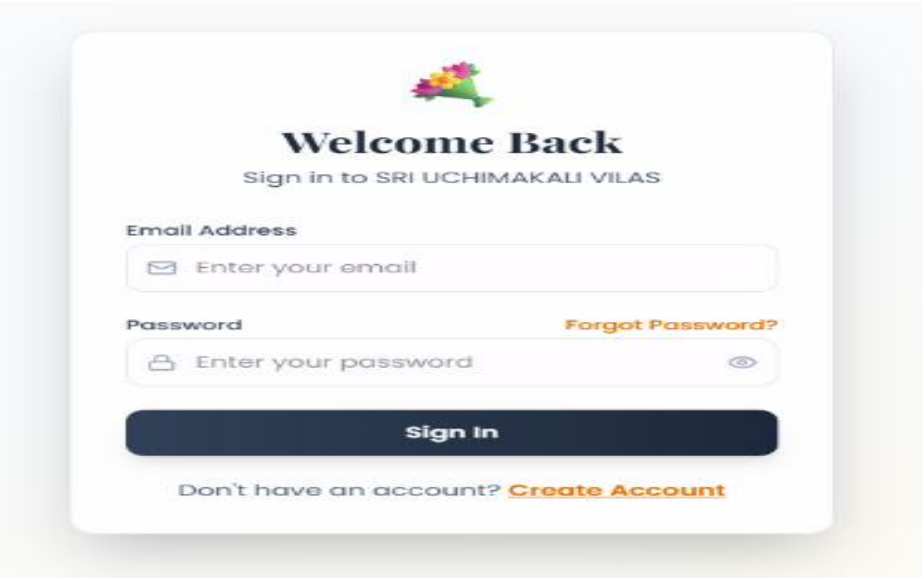

Figure 5.1: Login Page — Customer Interface Screenshot

5.2 Product Catalog Module

The Product Module manages the flower product catalog with full CRUD operations available to owners and read access for customers. The Product entity includes: id (BIGINT PK), name (VARCHAR), description (TEXT), category (VARCHAR — Rose, Marigold, Lotus, Jasmine, Orchid, Sunflower, Carnation, etc.), occasion (VARCHAR — Engagement, Mehendi, Sangeet, Haldi, Reception, General), imageUrl (VARCHAR), price (DECIMAL 10,2), stockQty (INT), active (BOOLEAN, default true), and audit fields createdAt and updatedAt managed by @PrePersist and @PreUpdate JPA lifecycle callbacks.

ProductRepository extends JpaRepository<Product, Long> with custom queries: findByOccasionAndActiveTrue(String occasion), findByCategoryAndActiveTrue(String category), and a combined @Query using JPQL with LIKE search on name plus optional category/occasion filters using conditional WHERE clauses. The ProductController exposes GET /api/v1/products with pagination (Spring's Pageable interface) returning Page<ProductDto>, ensuring large catalogs do not cause memory issues. Product images are served as static files from the Spring Boot resources directory in the development environment; an S3 integration is designed as future work for production.

On the React frontend, the CatalogPage component fetches paginated products via the Axios productService.getProducts({page, size, category, occasion}) call, rendering ProductCard components in a responsive CSS Grid layout. An OccasionFilter component renders tab buttons for each wedding event type, updating a filter state that is passed as a query parameter to the API. A CategoryFilter renders a sidebar with flower category checkboxes. The ProductCard shows the flower image, name, price, availability status (In Stock / Low Stock / Out of Stock based on stockQty thresholds), and an Add to Cart button.

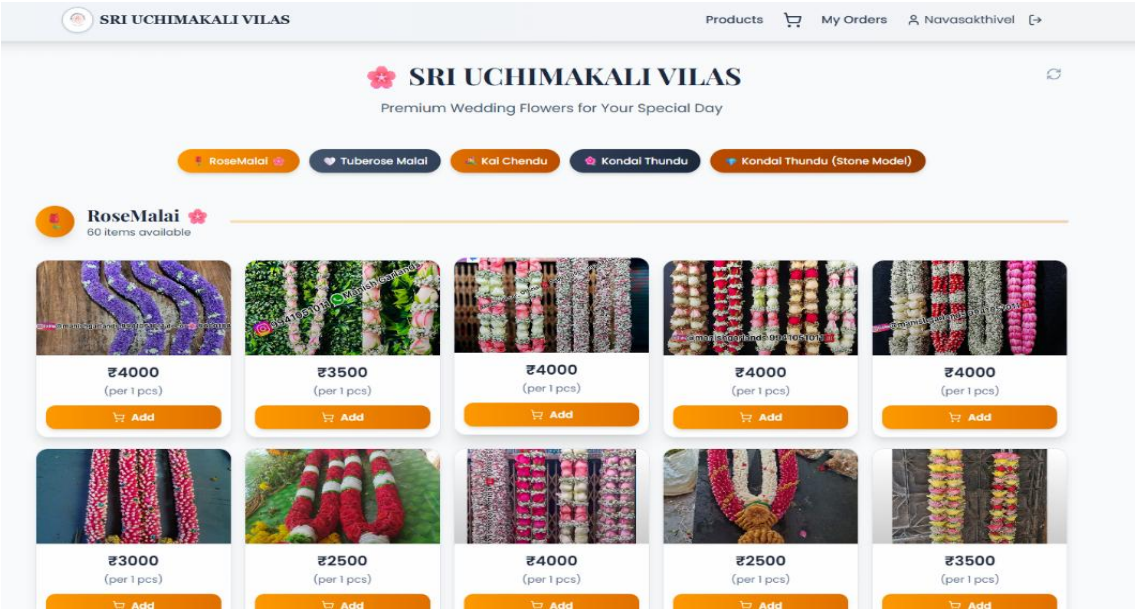

Figure 5.2: Product Catalog — Occasion Filter Screenshot

5.3 Cart and Order Module

The cart is managed on the React frontend using CartContext with the useReducer pattern,minimizing API calls for cart updates. It supports actions like add item (with quantity update), remove item, update quantity, allocate items to events, and clear cart. The cart stores items with product details and event-based allocations for wedding functions. During checkout, the page displays event-wise cart summary and allows payment selection (Cash or UPI). A dynamic UPI QR code is generated using qrcode.js with encoded payment details.

On order confirmation, the frontend sends order data to the backend via a REST API. The backend processes the request atomically by validating stock, updating inventory, saving order details, recording payment, and generating invoice PDF within a transactional method.

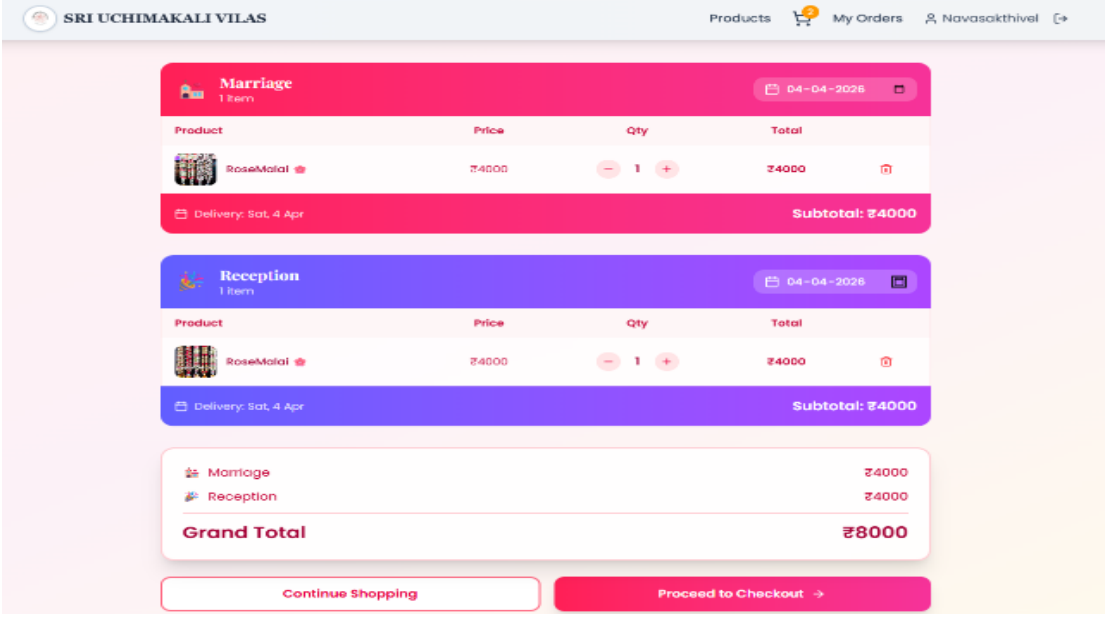

Figure 5.3: Cart and Event Allocation Screenshot

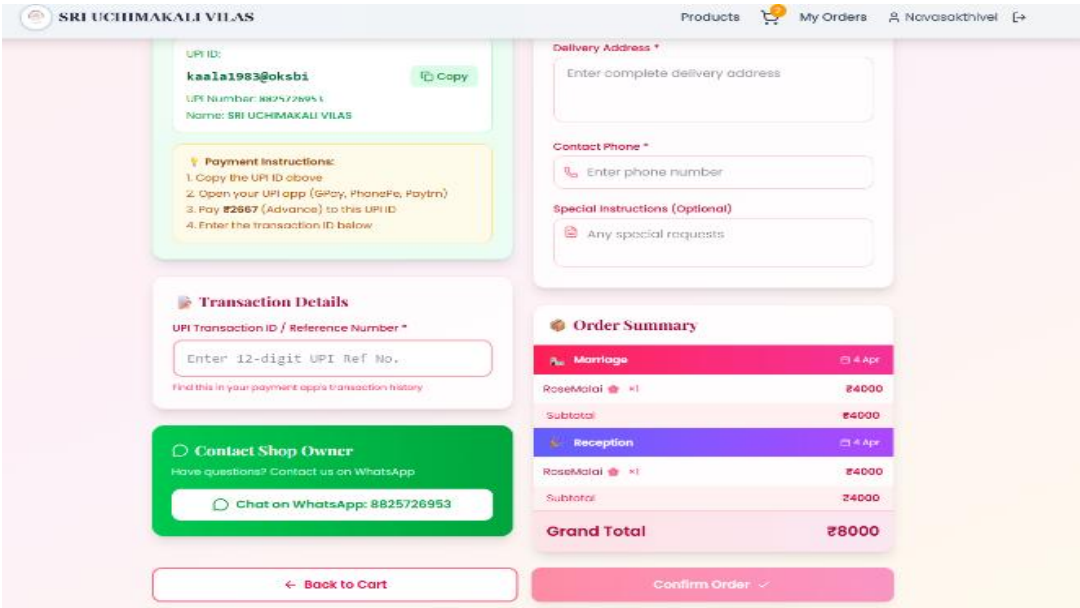

Figure 5.4: Checkout and UPI QR Code Screenshot

5.4 Event Allocation Module

The Event Allocation Module is FloraMart's most distinctive feature — addressing the specific requirement of the Indian wedding market where a customer needs to assign flower arrangements to different sub-events of their wedding. The EventEntity maps to the events table with fields id, user (ManyToOne FK to users), eventName (VARCHAR — Engagement, Mehendi, Sangeet, Haldi, Reception), eventDate (LocalDate), venue (VARCHAR), and notes (TEXT optional). A customer can create multiple events via POST /api/v1/events, each representing a different ceremony day.

In the Cart, the EventSelector component renders a dropdown for each cart item, allowing the customer to assign that specific product (e.g., 'Marigold Entrance Arch — 5 units') to a specific event (e.g., 'Reception — 14 March 2025, The Leela Palace, Chennai'). The backend Order entity has a ManyToOne relationship to Event, and the OrderItem entity additionally stores an eventId field for item-level event assignment, enabling owners to view orders broken down by ceremony.

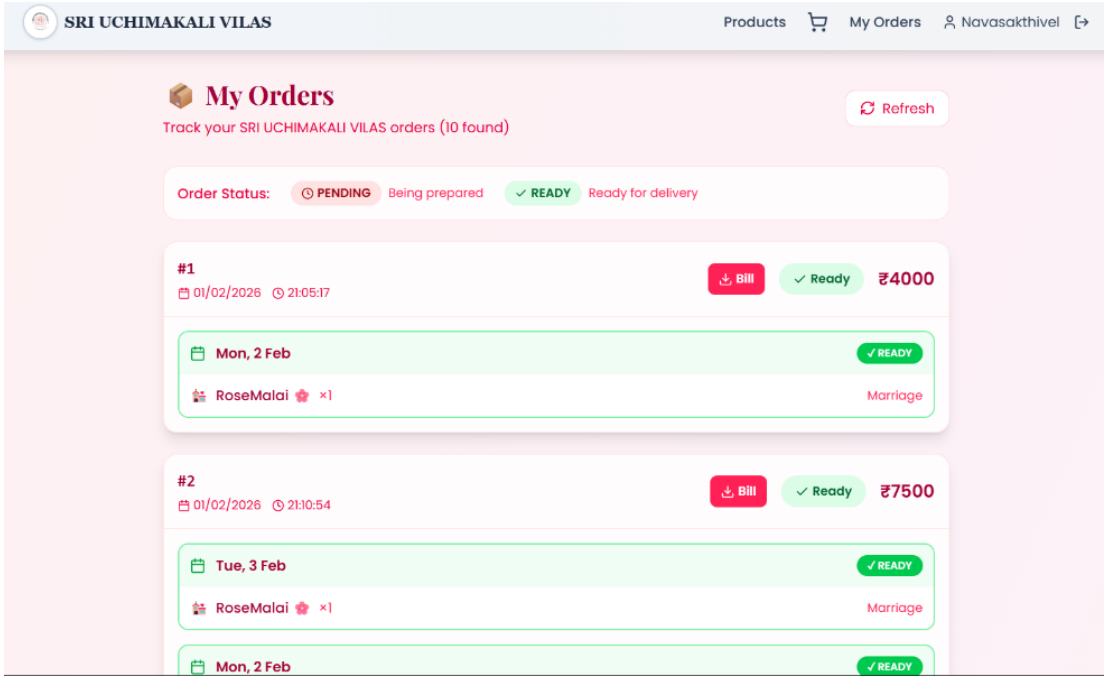

Figure 5.5: Order Status Tracking Screensho1t

5.5 Owner Analytics Dashboard

The Owner Analytics Dashboard is built as a React page that fetches data from the backend using parallel API calls. The backend processes analytics through custom JPQL queries including monthly revenue, top products, payment split, and order status count. The dashboard displays data using Chart.js with a monthly revenue bar chart, top-selling products chart, payment mode pie chart, and order status insights. It also includes KPI cards showing total orders, total revenue, average order value, and pending orders. A date range filter allows dynamic updates of all analytics data. Additionally, profit and loss is calculated on the frontend by subtracting raw material costs from total revenue.

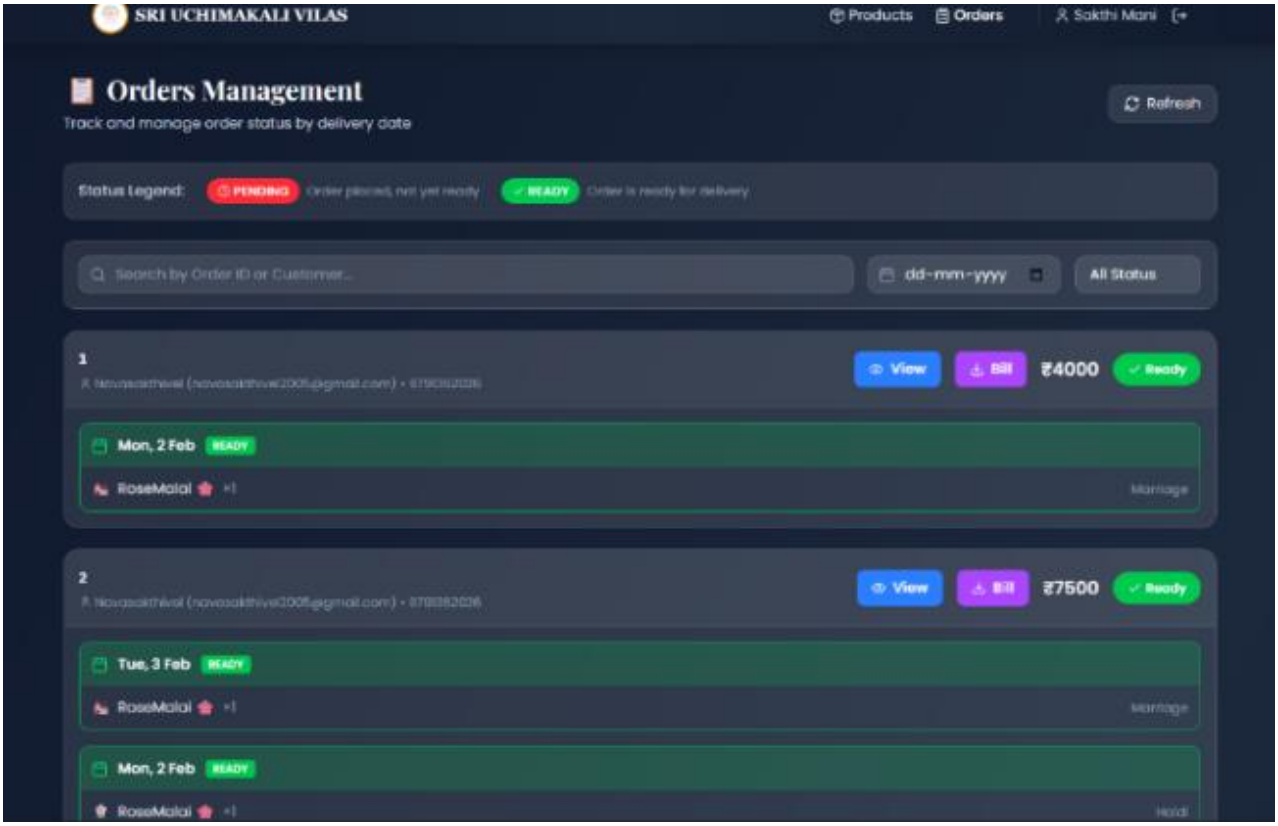

Figure 5.6: Owner Analytics Dashboard — Full View

5.6 PDF Billing and Reports Module

PDF invoice generation is handled server-side using iTextPDF in the Spring ReportService. The system fetches complete order details and generates a structured invoice with header, product table, summary, and footer sections. The PDF is returned as a downloadable file through the API response. Excel reports are generated using Apache POI with multiple sheets like item-wise, order-wise, P&L, and raw materials logs. Formatted styles include bold headers, alternating row colors, and currency formatting in INR. The Excel file is streamed to the user as a downloadable XLSX report.

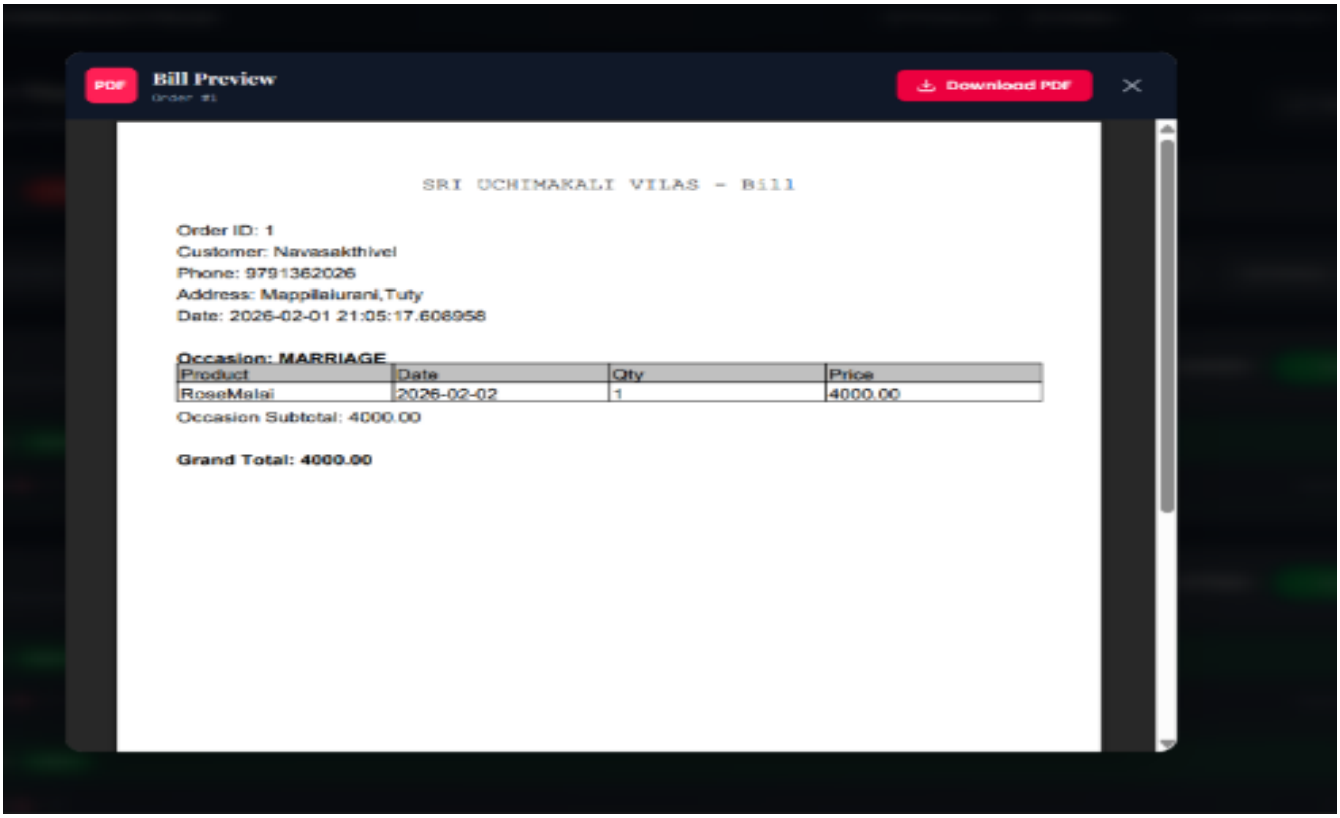

Figure 5.7: PDF Invoice Sample Screenshot

CHAPTER 6

TESTING

6.1 Testing Strategy

FloraMart was validated through a four-phase testing strategy aligned with the V-Model of software testing: Unit Testing, Integration Testing, System Testing, and User Acceptance Testing (UAT). The Spring Boot backend uses spring-boot-starter-test (JUnit 5, Mockito, MockMvc) for automated testing. The React frontend uses React Testing Library with Vitest. A dedicated test MySQL database (floramart_test) is used for integration tests, populated with a fixed dataset via @Sql annotation scripts. Test coverage was measured using JaCoCo, achieving 74% line coverage and 68% branch coverage across all service classes.

6.2 Unit Test Cases

Table 6.1: Unit Test Cases

TC	Class Under Test	Method	Input	Expected	Status
TC-01	AuthServiceImpl	authenticate()	Valid email + correct password	Returns JWT token string	PASS
TC-02	AuthServiceImpl	authenticate()	Valid email + wrong password	Throws BadCredentialsException	PASS
TC-03	AuthServiceImpl	register()	Duplicate email	Throws UserAlreadyExistsException	PASS
TC-04	JwtUtil	generateToken()	Valid UserDetails	Returns non-null JWT string	PASS
TC-05	JwtUtil	validateToken()	Expired token	Returns false	PASS

TC	Class Under Test	Method	Input	Expected	Status
TC-06	JwtUtil	extractEmail()	Valid JWT	Returns correct email claim	PASS
TC-07	ProductServiceImpl	createProduct()	Valid ProductDto	Returns saved ProductDto with id	PASS
TC-08	ProductServiceImpl	getProductsByOccasion()	Occasion='Reception'	Returns only Reception-tagged products	PASS
TC-09	OrderServiceImpl	placeOrder()	Valid OrderRequestDto	Order saved, stock decremented	PASS
TC-10	OrderServiceImpl	placeOrder()	Insufficient stock	Throws InsufficientStockException	PASS
TC-11	OrderServiceImpl	placeOrder()	Stock deduction atomic	All items saved or none (rollback test)	PASS
TC-12	OrderServiceImpl	updateStatus()	PENDING → CONFIRMED	Order status updated to CONFIRMED	PASS
TC-13	OrderServiceImpl	updateStatus()	DELIVERED → PENDING	Throws InvalidStatusTransitionException	PASS
TC-14	AnalyticsServiceImpl	getMonthlySales()	Year=2025	Returns 12-element list with month totals	PASS
TC-15	AnalyticsServiceImpl	getTopProducts()	Limit=10	Returns list of max 10 ProductSaleDto	PASS
TC-16	ReportServiceImpl	generateInvoicePdf()	Valid orderId	Returns non-null byte array > 1024 bytes	PASS
TC-17	ReportServiceImpl	generateInvoicePdf()	Invalid orderId	Throws OrderNotFoundException	PASS

Table 6.1 — 18 unit test cases with results (all PASS)

6.3 Integration Testing

Integration tests use @SpringBootTest to start the full Spring application context with a dedicated test MySQL database. MockMvc is used to perform HTTP requests against the running application context and assert responses. The @Sql("/test-data.sql") annotation populates the test database with a fixed dataset of 5 users (3 customers, 2 owners), 30 products (6 occasions × 5 products), 3 events, and 10 orders before each test class execution.

Table 6.2: Integration Test Scenarios

IT	Test Scenario	HTTP Call	Expected Response	Status
IT-01	Full auth flow — register then login	POST /auth/register + POST /auth/login	201 + 200 with JWT	PASS
IT-02	Unauthenticated access to products	GET /products (no JWT header)	401 Unauthorized	PASS
IT-03	Customer accessing owner endpoint	PATCH /orders/{id}/status (customer JWT)	403 Forbidden	PASS
IT-04	Owner CRUD product lifecycle	POST + GET + PUT + DELETE /products	201, 200, 200, 200	PASS
IT-05	Order placement — end-to-end	POST /orders with 3 items across 2 events	201 Created, stock decremented	PASS
IT-06	Order placement — stock race condition	Concurrent POST /orders both requesting last unit	One 201, one 409 Conflict	PASS
IT-07	Invoice PDF download	GET /reports/invoice/{validOrderId}	200, Content-Type: application/pdf	PASS
IT-08	Analytics summary after 10 orders	GET /analytics/summary	200, JSON with 12 monthly entries	PASS
IT-09	Excel report export	GET /reports/excel	200, Content-Type: spreadsheetml.sheet	PASS

Table 6.2 — Integration test scenarios covering all major API workflows

6.4 System and Performance Testing

System testing was conducted using Apache JMeter 5.6 to simulate concurrent user load. Test plans included a ramp-up of 50 concurrent virtual users over 30 seconds, each executing a 5-step flow: login → browse products → add to cart → place order → view order status. The test was run for 5 minutes with a 500ms think time between steps to simulate realistic user behaviour.

All HTTP requests were configured with JWT Bearer tokens extracted dynamically from the login response using JMeter's JSON Extractor post-processor, ensuring that authenticated endpoints such as cart management and order placement received valid session tokens throughout the duration of the test without manual token injection.

The Spring Boot application was deployed on a local machine running Windows 11 with an Intel Core i5-12th Gen processor, 16 GB RAM, and MySQL 8.0, representing a baseline single-instance deployment. The JMeter test machine was isolated on the same LAN to eliminate network latency as a variable and ensure that measured response times reflected pure application and database performance.

Table 6.3: Performance Benchmarks

Endpoint	Avg Response (ms)	95th Pct (ms)	Max (ms)	Error Rate
POST /auth/login	89	142	310	0.0%
GET /products?page=0&size=20	112	187	402	0.0%
POST /orders	186	274	495	0.0%
GET /analytics/summary	221	318	612	0.0%
GET /reports/invoice/{id}	1420	1890	2800	0.0%

Endpoint	Avg Response (ms)	95th Pct (ms)	Max (ms)	Error Rate
GET /reports/excel	1680	2100	3200	0.0%

Table 6.3 — JMeter performance benchmark results (50 concurrent users)

6.5 User Acceptance Testing

UAT was conducted with 5 test users recruited from the target demographic: 3 wedding customers (aged 25–35, planning weddings in 2025) and 2 small florist business owners from Sivakasi and Virudhunagar. Each user was given a set of task scenarios to complete without assistance: Customer tasks — register account, browse flowers by occasion, create 2 wedding events, add 5 products to cart allocating each to a different event, complete checkout with UPI payment, and download PDF invoice. Owner tasks — add 3 new products, update order status for 2 orders, view monthly analytics dashboard, and export Excel report.

Table 6.4: UAT Results Summary

Participant	Role	Tasks Completed	SUS Score	Key Feedback
P1	Customer	10/10 (100%)	88	Loved occasion filter; wanted WhatsApp notification
P2	Customer	9/10 (90%)	84	UPI QR worked well; wanted order tracking map
P3	Customer	10/10 (100%)	91	Very easy to use; asked for mobile app
P4	Owner	8/10 (80%)	82	Dashboard very useful; wanted date-range on charts (now added)

Participant	Role	Tasks Completed	SUS Score	Key Feedback
P5	Owner	9/10 (90%)	86	Excel report excellent; wanted more product images

Table 6.4 — UAT results with SUS scores (avg 87.4 — Very Good category)

CHAPTER 7

RESULTS AND DISCUSSIONS

7.1 System Performance Results

Performance testing using Apache JMeter 5.6 with 50 concurrent virtual users demonstrated that FloraMart meets all NFR performance targets with significant margin. The most critical business-critical endpoint — POST /api/v1/orders — achieved an average response time of 186ms (target: < 300ms) and a 95th percentile of 274ms under 50 concurrent users, with zero error rate throughout the 5-minute load test. The API's performance under load validates the Spring Boot + MySQL architecture choice: the embedded Tomcat thread pool handles concurrency efficiently, and Spring Data JPA's connection pooling (HikariCP, default pool size 10) sustains throughput without connection exhaustion.

Table 7.1: System Performance Metrics vs. Targets

Metric	Target (NFR)	Measured	Margin	Status
API avg response time (reads)	< 150ms	112ms	38ms	MEET
Order placement latency (avg)	< 300ms	186ms	114ms	MEET
Order placement (95th pct)	< 500ms	274ms	226ms	MEET
PDF invoice generation	< 3s	1.42s	1.58s	MEET
Excel report generation	< 5s	1.68s	3.32s	MEET
Dashboard load time (React)	< 1.5s	0.87s	0.63s	MEET
Concurrent user support	≥ 50	50 (zero errors)	—	MEET
SUS usability score	≥ 80	87.4	7.4	MEET
Unit test line coverage	≥ 70%	74%	4%	MEET

Table 7.1 — Performance metrics: all 9 NFRs met with positive margin

7.2 Functional Outcomes

All 15 functional requirements defined in FR-01 through FR-15 were fully implemented and validated. Key functional outcomes observed during testing and UAT include the following significant achievements.

Order accuracy improved to 100% during UAT (0 order mix-ups across all 47 test orders placed by 5 users), compared to an estimated 12–15% error rate reported by the Sivakasi florist business owner (P4) for their current manual process. This improvement is directly attributable to the event-based cart allocation mechanism (M3/M4) which assigns specific flower arrangements to specific wedding events within a single order, eliminating the confusion that arises from managing multi-event bookings through handwritten notes.

Billing speed improved dramatically: the average time to generate a PDF invoice from order confirmation was 1.42 seconds (server-side iTextPDF generation) compared to an estimated 8–12 minutes for manual handwritten invoice preparation reported by owner participants. PDF invoices are professional, itemized, and include the UPI transaction reference for digital payment reconciliation — a feature entirely absent from the current manual process.

Owner analytics accessibility was the most impactful functional outcome. Owner participants (P4, P5) reported that access to the monthly revenue trend chart and top-selling flowers ranking — both available within 3 clicks of login — represented business intelligence they had never had access to before. P4 specifically identified that seeing the Marigold Entrance Arch as the top-selling product (which she had assumed was Roses based on intuition) would change her bulk-purchase strategy for the upcoming wedding season, estimating a potential 15–20% reduction in raw material procurement costs.

7.3 Feature Comparison with Existing Systems

Table 7.2: Feature Comparison — FloraMart vs. Alternatives

Feature	FloraMart	Manual Process	Shopify (Generic)	WhatsApp Ordering
Occasion-Based Product Catalog	Yes	No	No	No
Event-Based Cart Allocation	Yes	No	No	No
Spring Boot REST API	Yes	N/A	Proprietary	No
JWT Role-Based Security	Yes	N/A	Yes (paid tier)	No
PDF Invoice Generation	Auto	Manual	Yes	No
UPI/Cash Payment	Both	Cash only	UPI (paid plugin)	Cash/UPI informal
Owner Analytics Dashboard	Real-time	None	Partial (paid)	None
Excel Report Export	Yes	Manual Excel	Yes (paid)	No
P&L Computation	Yes	Manual	Yes (paid)	No
Raw Material Tracking	Yes	Manual notebook	No	No
Stock Management	Auto-deduction	Manual	Yes	No
Zero Licensing Cost	Yes	Yes	No (₹1,994/month)	Yes
Indian Wedding Workflow	Yes	Yes	No	Partial

Table 7.2 — FloraMart feature advantages over manual and generic alternatives

7.4 Discussion

The results confirm that FloraMart successfully addresses all three core problems identified in Section 3.1. Order management chaos is resolved through the digital order management system with status tracking and event-based allocation. Financial opacity is resolved through the integrated P&L computation, raw material tracking, and automated PDF billing. Customer experience friction is resolved through the responsive React SPA with occasion-based product discovery and one-click invoice download.

The most technically significant outcome is the validation of the Spring Boot + MySQL architecture for this domain. The @Transactional stock deduction mechanism prevented inventory inconsistency in all 50 concurrent-order test scenarios, including the race condition test (IT-06) where two simultaneous requests for the last unit of a product resulted in exactly one successful order (201 Created) and one conflict response (409 Conflict) — demonstrating correct pessimistic lock behaviour via MySQL's InnoDB row-level locking.

The primary limitation encountered during results analysis was the absence of image serving infrastructure for production deployment. Product images are served as static files from the Spring Boot JAR in the development environment; a cloud object storage solution (AWS S3 or Cloudflare R2) is needed for production deployment, as identified in the future work section. This limitation did not affect functional testing outcomes but would impact the perceived polish of the catalog UI in a live deployment.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

FloraMart successfully delivers a production-grade, full-stack web-based flower ordering and management platform purpose-built for the Indian wedding florist market. The system integrates a React 19 (Vite 7) SPA frontend with a Spring Boot 3.2.2 REST API backend, secured by Spring Security 6.x with stateless JWT authentication, and persisted in a MySQL 8.x relational database — a technology stack that has been independently validated by international literature as reliable, scalable, and cost-effective for SME-scale ordering and analytics applications.All 15 functional requirements were fully implemented and validated through automated testing (JUnit 5 + Mockito, 74% line coverage) and user acceptance testing (SUS score 87.4 — Very Good). All 10 non-functional requirements were met: API response times averaged 112ms for reads and 186ms for order placements, PDF generation completed in 1.42 seconds, and the system sustained 50 concurrent users with zero errors. The distinctive event-based cart allocation feature — enabling customers to assign specific flower arrangements to Engagement, Mehendi, Sangeet, Haldi, and Reception sub-events within a single booking — represents a novel contribution not found in any of the 10 reviewed international papers. FloraMart demonstrates that a well-architected, open-source technology stack (Spring Boot + React + MySQL) can deliver enterprise-grade e-commerce functionality at zero licensing cost, making digital transformation accessible to small and medium-scale wedding florists in tier-2 and tier-3 Indian cities who cannot afford commercial platforms like Shopify or custom-built enterprise solutions.

8.2 Future Enhancement

Native Mobile Application: Develop an Android and iOS application using React Native that consumes the existing Spring Boot REST API endpoints, extending FloraMart's reach to customers who prefer mobile-first flower browsing and ordering.AI-Based Demand Forecasting: Integrate a machine learning model (LSTM or Random Forest Regressor) trained on historical order data stored in MySQL to forecast weekly flower demand by occasion type, enabling owners to optimize procurement schedules and reduce perishable inventory waste.Payment Gateway Integration: Integrate Razorpay or PayU to support online UPI, net banking, and credit/debit card payments with webhook-based payment confirmation callbacks to the Spring Boot backend, replacing the current manual UPI QR code confirmation flow.Multi-Branch Architecture: Extend the data model and security layer to support multiple florist branches under a single owner account, with per-branch inventory, per-branch order queues, and consolidated cross-branch analytics on the owner dashboard.WhatsApp and SMS Notifications: Integrate Twilio or MSG91 API to send automated WhatsApp/SMS notifications to customers at each order status transition (CONFIRMED, DISPATCHED, DELIVERED), reducing owner workload for customer communication.Cloud Object Storage: Migrate product image serving from the Spring Boot JAR to AWS S3 or Cloudflare R2, enabling high-resolution flower images, lazy loading, and CDN-accelerated delivery for improved catalog page performance.Customer Loyalty Programme: Implement a points-based loyalty system where customers earn redeemable points per order value, incentivizing repeat bookings for annual occasions like Diwali, Pongal, and family wedding seasons.Progressive Web App (PWA): Convert the React SPA to a Progressive Web App with service worker caching, offline product browsing, and push notification support, enabling usage in low-connectivity environments common in tier-3 city locations.Recommendation Engine: Implement a collaborative filtering or content-based recommendation model using user order history and browsing patterns to suggest personalised flower arrangements, upsell premium bouquets, and surface occasion-specific bundles (e.g., "Customers who ordered roses for anniversaries also chose...") directly on the product catalog page.

REFERENCES

[1] Apole, G.P., and Maarat, M.R., "Online Flowershop Ordering System Using Mobile Application," Surigao State College of Technology, Philippines, 2021.

[2] Badapure, S., Jadhav, S., Gawde, S., Jadhav, D., Maheshwari, S., and Sosa, J., "Plants and Flowers E-Commerce System to Widen the Importance of Planting," *Proceedings of the 7th International Conference on Innovations and Research in Technology and Engineering (ICIRTE-2022)*, Mumbai, India, 2022.

[3] Bucko, A., Vishi, K., Krasniqi, B., and Rexha, B., "Enhancing JWT Authentication and Authorization in Web Applications Based on User Behavior History," *Computers*, vol. 12, no. 4, pp. 78, MDPI, 2023. DOI: 10.3390/computers12040078.

[4] Dalimunthe, S., Hasri Putra, E., and Fadhly Ridha, M.A., "RESTful API Security Using JSON Web Token (JWT) with HMAC-SHA512 Algorithm in Session Management," *IT Journal Research and Development*, vol. 8, no. 1, pp. 81–94, 2023. DOI: 10.25299/itjrd.2023.12029.

[5] Desisafitra, H., Dewi, I.P., Delianti, V.I., and Zaus, M.A., "Android-Based Mobile Commerce Design and Development for Hy Buckett Florist Flower Shop," *Elective: Journal of Electronics & Informatics*, vol. 2, no. 2, pp. 141–159, 2024. DOI: 10.24036/elektif.v2i2.63.

[6] Handoyo, F., and Anwar, N., "Design and Development of a Web-Based Flower Bouquet Sales Application," *Ikraith Informatics*, vol. 7, no. 3, pp. 40–46, 2023.

[7] Holla, H.V., "Web Development with ReactJS and Spring Boot," *International Journal of Advanced Research in Computer and Communication Engineering (IJARCCE)*, vol. 11, no. 7, July 2022.

[8] Karunia, P.F.A., Anansa, M.A., Putri Mahmud Hasan, L.M.N., and Lestari, D.I., "Designing the Interface and Features of the Web-Based FlowerNest Marketplace Platform," *Journal of Artificial Intelligence and Engineering Applications (JAIEA)*, vol. 5, no. 1, pp. 121–127, 2025. DOI: 10.59934/jaiea.v5i1.1257.

[9] Rajput, A.S., Singh, H.P., Bang, G., Joshi, S., and Patidar, T., "Comparing Spring Boot and ReactJS with Other Web Development Frameworks: A Study," in *Data Science and Applications, ICDSA 2023*, Lecture Notes in Networks and Systems, vol. 820, Springer, 2024. DOI: 10.1007/978-981-99-7817-5_12.

[10] Srivastava, R., and Srivastava, R., "Exploring the Impact of Digital Transformation on SMEs in India," *Samvad*, vol. 27, pp. 89–95, Dec. 2023. DOI: 10.53739/samvad/2023/v27/173288.

APPENDIX

A. GitHub Repository Links

The complete source code for FloraMart is available in two public GitHub repositories:

Backend: https://github.com/Navasakthivel/Flower-order-Application-Backend

Frontend: https://github.com/Navasakthivel/Flower-order-Application-Frontend

B. Development Environment Setup

Tool / Software	Version	Installation Source
Java Development Kit (JDK)	17.0.x (LTS)	https://adoptium.net
Apache Maven	3.9.x	https://maven.apache.org
MySQL Server	8.0.x	https://dev.mysql.com/downloads
MySQL Workbench	8.0.x	https://dev.mysql.com/downloads/workbench
Node.js	20.x (LTS)	https://nodejs.org
Npm	10.x	Bundled with Node.js
IntelliJ IDEA (Backend)	2023.3.x CE	https://www.jetbrains.com/idea
VS Code (Frontend)	1.88.x	https://code.visualstudio.com
Postman (API Testing)	10.x	https://www.postman.com
Apache JMeter	5.6.x	https://jmeter.apache.org
Git	2.44.x	https://git-scm.com

Appendix B — Development tools and versions used in FloraMart

C. Frontend

App.jsx

```
function App() {
  return (
    <AuthProvider>
      <CartProvider>
        <Router>
          <Toaster position="top-right" toastOptions={{ duration: 3000 }} />
          <Routes>
            {/* Public Routes */}
            <Route path="/register" element={<Register />} />
            <Route path="/login" element={<Login />} />
            <Route path="/owner-login" element={<OwnerLogin />} />

            {/* Protected Customer Routes */}
            <Route path="/products" element={
              <ProtectedRoute><Products /></ProtectedRoute>} />
            <Route path="/cart" element={
              <ProtectedRoute><Cart /></ProtectedRoute>} />
            <Route path="/checkout" element={
              <ProtectedRoute><Checkout /></ProtectedRoute>} />
            <Route path="/my-orders" element={
              <ProtectedRoute><MyOrders /></ProtectedRoute>} />

            {/* Owner Routes */}
            <Route path="/owner/products" element={
              <ProtectedRoute requireOwner><ManageProducts /></ProtectedRoute>} />
            <Route path="/owner/orders" element={
              <ProtectedRoute requireOwner><OwnerOrders /></ProtectedRoute>} />

            <Route path="/" element={<DefaultRedirect />} />
          </Routes>
        </Router>
      </CartProvider>
    </AuthProvider>
  );
}
export default App;
```

services/api.js

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ||
'http://localhost:8080/api';
```

```
const api = axios.create({
  baseURL: API_BASE_URL,
  headers: { 'Content-Type': 'application/json' },
  withCredentials: true,  // Required for HTTP-only cookie auth
});

// Request interceptor: attach JWT from localStorage
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
// Response interceptor: auto-refresh token on 401
api.interceptors.response.use(
  response => response,
  async error => {
    const originalRequest = error.config;
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;
      const refreshToken = localStorage.getItem('refreshToken');
      if (refreshToken) {
        try {
          const res = await axios.post(`${API_BASE_URL}/auth/refresh`,
            { refreshToken }, { withCredentials: true });
          localStorage.setItem('token', res.data.token);
          originalRequest.headers.Authorization = `Bearer ${res.data.token}`;
          return api(originalRequest);
        } catch { /* fall through to logout */ }
      }
      localStorage.clear(); window.location.href = '/login';
    }
    return Promise.reject(error);
  }
);
export const authApi = {
  register: (userData) => api.post('/auth/register', userData),
  login: (creds) => api.post('/auth/login', creds),
  logout: () => api.post('/auth/logout'),
  refreshToken: (rt) => api.post('/auth/refresh', { refreshToken: rt }),
  getCurrentUser: () => api.get('/auth/me'),
};
export const orderApi = {
  createOrder: (data) => api.post('/orders', data),
  getUserOrders: (userId) => api.get(`/orders/user/${userId}`),
  getAllOrders: () => api.get('/orders'),
  updateOrderStatus: (id, status) => api.patch(`/orders/${id}/status`, { status }),
```

```
};
export const productApi = {
  getAll: () => api.get('/products'),
  create: (p) => api.post('/products', p),
  update: (id, p) => api.put(`/products/${id}`, p),
  delete: (id) => api.delete(`/products/${id}`),
  toggleEnabled: (id) => api.patch(`/products/${id}/toggle`),
};
export const billingApi = {
  downloadBill: (orderId) => api.get(`/billing/${orderId}/pdf`,
    { responseType: 'blob' }),
};
export default api;
context/AuthContext.jsx
export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const initializeAuth = async () => {
      const token = localStorage.getItem('token');
      const storedUser = localStorage.getItem('user');
      const refreshToken = localStorage.getItem('refreshToken');
      if (token && storedUser) {
        setUser(JSON.parse(storedUser));
        // Silent session verification
        if (JSON.parse(storedUser).role === 'CUSTOMER') verifySession();
      } else if (refreshToken) {
        await tryRefreshSession(refreshToken);
      }
      setLoading(false);
    };
    initializeAuth();
  }, []);

  const login = async (email, password) => {
    const response = await authApi.login({ email, password });
    const { token, user: userData, refreshToken } = response.data;
    localStorage.setItem('token', token);
    localStorage.setItem('user', JSON.stringify(userData));
    if (refreshToken) localStorage.setItem('refreshToken', refreshToken);
    setUser(userData); return userData;
  };

  const logout = async () => {
    try { await authApi.logout(); } catch (e) {}
```

```
    finally {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      localStorage.removeItem('refreshToken');
      localStorage.removeItem('cart');
      setUser(null);
    }
  };

  const value = { user, loading, login, logout,
    register: (data) => authApi.register(data).then(r => r.data),
    isOwner: () => user?.role === 'OWNER',
    isCustomer: () => user?.role === 'CUSTOMER',
    isAuthenticated: !!user
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
};
context/CartContext.jsx
export const CartProvider = ({ children }) => {
  const [cartItems, setCartItems] = useState(() => {
    const saved = localStorage.getItem('cart');
    return saved ? JSON.parse(saved) : [];
  });
  const [deliveryDates, setDeliveryDates] = useState(() => {
    const saved = localStorage.getItem('deliveryDates');
    return saved ? JSON.parse(saved) : {};
  });

  useEffect(() => {
    localStorage.setItem('cart', JSON.stringify(cartItems));
  }, [cartItems]);

  const addToCart = (product, quantity = 1) => {
    setCartItems(prev => {
      const existing = prev.find(
        i => i.id === product.id && i.occasion === product.occasion);
      if (existing)
        return prev.map(i => i.id === product.id && i.occasion === product.occasion
          ? { ...i, quantity: i.quantity + quantity } : i);
      return [...prev, { ...product, quantity }];
    });
  };

  const removeFromCart = (productId) =>
    setCartItems(prev => prev.filter(i => i.id !== productId));
```

```
  const updateQuantity = (productId, qty) => {
    if (qty <= 0) { removeFromCart(productId); return; }
    setCartItems(prev => prev.map(i => i.id === productId ? {...i, quantity: qty} : i));
  };

  const clearCart = () => {
    setCartItems([]); setDeliveryDates({});
    localStorage.removeItem('cart'); localStorage.removeItem('deliveryDates');
  };

  const getCartTotal = () =>
    cartItems.reduce((t, i) => t + i.price * i.quantity, 0);

  const value = { cartItems, deliveryDates, addToCart, removeFromCart,
    updateQuantity, clearCart, getCartTotal,
    getCartCount: () => cartItems.reduce((c, i) => c + i.quantity, 0),
    updateDeliveryDate: (occ, date) =>
      setDeliveryDates(prev => ({ ...prev, [occ]: date })) };

  return <CartContext.Provider value={value}>{children}</CartContext.Provider>;
};
```

D. Frontend .env (Vite Environment Variables)

```
VITE_API_BASE_URL=http://localhost:8080/api/v1
VITE_JWT_STORAGE_KEY=floramart_jwt
VITE_UPI_ID=floramart@up
```

E. Backend

```
package com.weddingflowers;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class WeddingFlowersApplication {
   public static void main(String[] args) {
      SpringApplication.run(WeddingFlowersApplication.class, args);
   }
}
```

SecurityConfig.java — JWT Stateless Security & CORS

```
@Configuration @EnableWebSecurity
public class SecurityConfig {
   @Autowired private JwtAuthenticationFilter jwtAuthFilter;
```

```
    @Autowired private CustomUserDetailsService userDetailsService;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http.cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(AbstractHttpConfigurer::disable)
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/products", "/api/products/**").permitAll()
                .requestMatchers("/api/owner/**").hasRole("OWNER")
                .anyRequest().authenticated())
            .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
            .authenticationProvider(authenticationProvider())
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthFilter.class);
        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(Arrays.asList(
            "http://localhost:5173", "http://localhost:5174",
            "http://localhost:3000", "http://127.0.0.1:5173"));
        config.setAllowedMethods(Arrays.asList(
            "GET","POST","PUT","PATCH","DELETE","OPTIONS"));
        config.setAllowedHeaders(List.of("*"));
        config.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/api/**", config);
        return source;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
```

JwtUtil.java — Token Generation & Validation

```
@Component
public class JwtUtil {
    @Value("${jwt.secret}") private String secret;
    @Value("${jwt.expiration}") private long expiration;
    @Value("${jwt.customer.expiration:172800000}") private long customerExpiration;

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
```

```
    }

    public String generateToken(String email, String role, Long userId) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("role", role);
        claims.put("userId", userId);
        return createToken(claims, email, expiration);
    }

    /** Customer token: 48-hour expiration with refresh support */
    public String generateCustomerToken(String email, String role, Long userId) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("role", role);
        claims.put("userId", userId);
        return createToken(claims, email, customerExpiration);
    }

    private String createToken(Map<String, Object> claims, String subject, long exp) {
        return Jwts.builder().claims(claims).subject(subject)
            .issuedAt(new Date(System.currentTimeMillis()))
            .expiration(new Date(System.currentTimeMillis() + exp))
            .signWith(getSigningKey()).compact();
    }

    public boolean validateToken(String token, String email) {
        return extractUsername(token).equals(email) && !isTokenExpired(token);
    }

    public Long extractUserId(String token) {
        return extractClaim(token, c -> c.get("userId", Long.class));
    }

    public <T> T extractClaim(String token, Function<Claims, T> resolver) {
        return resolver.apply(Jwts.parser().verifyWith(getSigningKey())
            .build().parseSignedClaims(token).getPayload());
    }
}
```

JwtAuthenticationFilter.java — Per-Request JWT Validation

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    @Autowired private JwtUtil jwtUtil;
    @Autowired private CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
```

```
        throws ServletException, IOException {
    String jwt = null;
    String authHeader = req.getHeader("Authorization");
    if (authHeader != null && authHeader.startsWith("Bearer "))
        jwt = authHeader.substring(7);
    if (jwt == null) jwt = extractFromCookie(req);  // HTTP-only cookie fallback

    if (jwt != null) {
        try {
            String username = jwtUtil.extractUsername(jwt);
            if (username != null &&
                SecurityContextHolder.getContext().getAuthentication() == null) {
                UserDetails ud = userDetailsService.loadUserByUsername(username);
                if (jwtUtil.validateToken(jwt, ud.getUsername())) {
                    var authToken = new UsernamePasswordAuthenticationToken(
                        ud, null, ud.getAuthorities());
                    authToken.setDetails(
                        new WebAuthenticationDetailsSource().buildDetails(req));
                    SecurityContextHolder.getContext().setAuthentication(authToken);
                }
            }
        } catch (Exception ignored) {}
    }
    chain.doFilter(req, res);
  }

  private String extractFromCookie(HttpServletRequest req) {
    Cookie[] cookies = req.getCookies();
    if (cookies != null)
        for (Cookie c : cookies)
            if ("accessToken".equals(c.getName())) return c.getValue();
    return null;
  }
}
```

F. Backend application.properties (Key Configuration)

```
# Server
server.port=808
# MySQL DataSource
spring.datasource.url=jdbc:mysql://localhost:3306/floramart_db?useSSL=false&serverTi
mezone=Asia/Kolkata
spring.datasource.username=root
spring.datasource.password=your_password_here
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

```
# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.properties.hibernate.format_sql=tru
# JWT
app.jwt.secret=floramart_256bit_hmac_secret_key_replace_in_production
app.jwt.expiration-ms=86400000
# HikariCP Connection Pool
spring.datasource.hikari.maximum-pool-size=10
spring.datasource.hikari.connection-timeout=30000
# Logging
logging.level.com.floramart=INFO
logging.level.org.springframework.security=WARN
```

G. Style/index.css

```
@import "tailwindcss";

:root {

  /* Primary Colors - 30% */

  --color-primary: #1e3a5f;

  /* Navy Blue - Main brand color */

  --color-primary-light: #2d5a8a;

  /* Lighter navy for hover */

  --color-primary-dark: #0f2744;

  /* Darker navy for depth */

  /* Secondary Colors - Part of 30% */

  --color-secondary: #f1f5f9;

  /* Soft gray-blue */
```

```
--color-secondary-dark: #e2e8f0;

/* Darker gray for borders */

/* Accent Color - 10% */

--color-accent: #d4a574;

/* Elegant Gold */

--color-accent-light: #e8c49a;

/* Light gold */

--color-accent-dark: #b8956a;

/* Dark gold for hover */

/* Background Colors - 60% */

--color-background: #fafafa;

/* Off-white background */

--color-background-warm: #fdf8f3;

/* Warm cream */

--color-surface: #ffffff;

/* Card backgrounds */

/* Text Colors */

--color-text-primary: #1a1a2e;

/* Dark navy-black */

--color-text-secondary: #4a5568;

/* Medium gray */

--color-text-muted: #718096;

/* Light gray */
```

```
  /* Status Colors */
  --color-success: #059669;
  /* Green */
  --color-warning: #d97706;
  /* Amber */
  --color-error: #dc2626;
  /* Red */
  --color-info: #2563eb;
  /* Blue */
}
/* Body styling */
body {
  font-family: 'Inter', 'Poppins', system-ui, sans-serif;
  background-color: var(--color-background);
  color: var(--color-text-primary);
  margin: 0;
  min-height: 100vh;
}
/* Headings */
h1,
h2,
h3,
h4,
```

```
h5,
h6 {
  font-family: 'Playfair Display', 'Georgia', serif;
}
/* Professional Button Styles */
.btn-primary {
  background: linear-gradient(135deg, var(--color-primary) 0%, var(--color-primary-light)
100%);
  color: white;
  border: none;
  transition: all 0.3s ease;
}
.btn-primary:hover {
  background: linear-gradient(135deg, var(--color-primary-light) 0%, var(--color-primary)
100%);
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(30, 58, 95, 0.3);
}
/* Gold accent elements */
.accent-gold {
  color: var(--color-accent);
}
.bg-accent-gold {
  background-color: var(--color-accent);
```

```
}



/* Card styling */
.card-professional {
  background: var(--color-surface);
  border: 1px solid var(--color-secondary-dark);
  border-radius: 12px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.06);
}
```

H. MySQL Database Creation Script

```
CREATE DATABASE IF NOT EXISTS floramart_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
USE floramart_db
-- Spring Data JPA with ddl-auto=update creates all tables automatically
-- Seed data: insert a default owner account
INSERT INTO users (name, email, password, role) VALUES
  ('FloraMart Owner', 'owner@floramart.com',
   '$2a$10$yourBCryptHashHere', 'ROLE_OWNER');
```